



## **Copyright Statement**

This copy was made pursuant to the [Fair Dealing Advisory of the University of Waterloo](#) and/or is covered and authorized by a license granted by the owners of the copyright on the works reproduced therein.

The copy may only be used for the purpose of research, private study, criticism, review, news reporting, education, satire or parody. If the copy is used for the purpose of review, criticism, or news reporting, the source and the name of the author must be mentioned. The use of this copy for any other purpose may constitute violation of Canadian Copyright Law, and/or relevant licensing agreements.

The Structure of the Eh Compiler: Code Generation

by

G. Bert Bonkowski

PROPERTY OF THE LIBRARY  
UNIVERSITY OF WATERLOO

A thesis  
presented to the University of Waterloo  
in partial fulfillment of the requirements  
for the degree of  
Master of Mathematics  
in  
Computer Science

Waterloo, Ontario 1978

© G. Bert Bonkowski, 1978

I hereby declare I am the sole author of this thesis.

I authorize the University of Waterloo to lend this thesis to other institutions or individuals for the purpose of scholarly research.

A handwritten signature in cursive script, reading "G. Brent Bonkowski".

I further authorize the University of Waterloo to reproduce this thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

A handwritten signature in cursive script, reading "G. Brent Bonkowski".

The University of Waterloo requires the signatures of all persons using or photocopying this thesis, Please sign below and give address and date.



## Abstract

The Eh compiler is designed to be portable over a large set of machines. It currently runs on two dissimilar computers and produces relocatable load code for four computers with quite different architectures and operating environments. The language compiled by the Eh compiler is the base language for the portable real-time operating system Thoth, developed at the University of Waterloo.

The structure of the code generation phases of the Eh compiler is presented with emphasis placed on interactions between a set of support functions and a set of functions for generating machine instructions. A novel method for generating code for expressions using tables is described, as well as a structure for generating code which depends on the distance between an instruction and its operand.

The structure of the code generation phases allows easy access to areas which depend on the machine, thus making it easier to convert the compiler to generate code for a new machine. The structure is also useful for developing algorithms for code generation or register allocation. Porting the compiler has involved two to four man-months of effort depending on the complexity of the machine and the experience of the people involved.

### Acknowledgements

This work would not have been possible without the support and direction provided by Prof. Michael Malcolm. A number of people have contributed to the development of the Eh compiler. Included in this group are Mike Afheldt, Otmar Bocharadt, Reinaldo Braga, Dave Bryant, Morven Gentleman, Sam Henning, Tom Miller, Alfredo Piquer, Patricio Poblete, Laurie Rapsey, Gary Sager, Gary Stafford, Ian Telford and Fred Young. John Corman maintained the hardware. A contributor in a non-technical capacity has been my wife Esti, without whom I would surely be lost in the chasms of computing.

## Table of contents

|   |                                                |     |
|---|------------------------------------------------|-----|
| 1 | Introduction . . . . .                         | 1   |
|   | 1.1 Compiler Phases . . . . .                  | 4   |
|   | 1.2 Eh Language Domain . . . . .               | 7   |
|   | 1.3 Porting the Eh Compiler . . . . .          | 8   |
| 2 | Load Code Format . . . . .                     | 11  |
|   | 2.1 Working Symbols . . . . .                  | 12  |
|   | 2.2 Relocation Base Registers . . . . .        | 12  |
|   | 2.3 Relocation Descriptors . . . . .           | 13  |
|   | 2.4 The Working Register . . . . .             | 14  |
|   | 2.5 Load Code Support Functions . . . . .      | 14  |
|   | 2.6 Load Code Example . . . . .                | 16  |
| 3 | Code Generation For External Modules . . . . . | 18  |
|   | 3.1 Manifests . . . . .                        | 19  |
|   | 3.2 Data Segments . . . . .                    | 21  |
|   | 3.3 Counter Stack . . . . .                    | 24  |
|   | 3.4 Reference Table . . . . .                  | 25  |
|   | 3.5 Support Functions . . . . .                | 26  |
|   | 3.6 Portability . . . . .                      | 31  |
| 4 | Code Generation For Function Modules . . . . . | 32  |
|   | 4.1 Input And Overall Structure . . . . .      | 34  |
|   | 4.1.1 Declaration Statements . . . . .         | 35  |
|   | 4.1.2 Expression Statements . . . . .          | 39  |
|   | 4.1.3 Select Statements . . . . .              | 42  |
|   | 4.1.4 Miscellaneous Statements . . . . .       | 44  |
|   | 4.2 Manifests . . . . .                        | 46  |
|   | 4.3 The Code Function . . . . .                | 49  |
|   | 4.4 Stack Support . . . . .                    | 52  |
|   | 4.5 Select Statement Support . . . . .         | 57  |
|   | 4.6 Expressions . . . . .                      | 62  |
|   | 4.6.1 Expression Tree Modification . . . . .   | 62  |
|   | 4.6.2 Register Allocation . . . . .            | 68  |
|   | 4.6.3 Machine Instruction Generation . . . . . | 71  |
|   | 4.7 Control-path Support . . . . .             | 87  |
|   | 4.7.1 Defer Queue . . . . .                    | 93  |
|   | 4.7.2 Deferring Instructions . . . . .         | 95  |
|   | 4.7.3 Backplugging Instructions . . . . .      | 99  |
|   | 4.7.4 Checking Symbol Addressability . . . . . | 100 |
|   | 4.8 Portability . . . . .                      | 108 |
| 5 | Conclusions . . . . .                          | 110 |
|   | 5.1 Future Research . . . . .                  | 113 |
|   | Bibliography. . . . .                          | 116 |

|            |                                           |     |
|------------|-------------------------------------------|-----|
| Appendix A | Code and Data Input . . . . .             | 117 |
| A.1        | Externals_parsil . . . . .                | 117 |
| A.2        | Cleanil . . . . .                         | 118 |
| Appendix B | Code table functions for TI 990 . . . . . | 122 |
| Appendix C | Stacks on the TI 990. . . . .             | 136 |
| Appendix D | Eh Implementation on the Nova . . . . .   | 141 |

## List of Illustrations

|                                                 |    |
|-------------------------------------------------|----|
| 1.1 Compiler structure. . . . .                 | 5  |
| 2.1 Example load code . . . . .                 | 17 |
| 3.1 Example vector external . . . . .           | 22 |
| 3.2 Data segment usage. . . . .                 | 23 |
| 3.3 Example Data output . . . . .               | 30 |
| 4.1 Symbol table entry. . . . .                 | 35 |
| 4.2 Symbol type descriptor. . . . .             | 37 |
| 4.3 Average comparisons per module. . . . .     | 39 |
| 4.4 Expression tree node. . . . .               | 41 |
| 4.5 Modification of function arguments. . . . . | 64 |
| 4.6 Nested function calls . . . . .             | 66 |
| 4.7 Modification of function calls. . . . .     | 67 |
| 4.8 Arith_expr_gen select case example. . . . . | 76 |
| 4.9 Code table example. . . . .                 | 77 |
| 4.10 Code table test entry. . . . .             | 78 |
| 4.11 Code table action entry. . . . .           | 79 |
| 4.12 Defer queue entry. . . . .                 | 93 |

## Chapter 1

### Introduction

Books and papers on compiler construction (with the exception of Wulf (1975)) are usually concerned with certain algorithms in a compiler rather than its overall structure. Implementation details are generally too involved to present fully, so the more interesting aspects of input scanning (sometimes called lexical scanning), syntax analysis (parsing) and code generation are presented. The reader is usually expected to extract the ideas presented in the literature and apply them to his or her own particular requirements. Unfortunately, the synthesis of such ideas can prove difficult.

This thesis describes the structure of the code generation phases of the Eh compiler developed at the University of Waterloo. The Eh compiler is implemented in the Eh language, described by Braga (1976), which is the base language for the portable real-time operating system, Thoth (Cheriton, et al., 1978). Eh is a descendant of the B language, developed at Bell Laboratories (Johnson, 1973). The B language is a descendant of BCPL (Richards, 1969). Considerable attention is given to the interaction between

machine-invariant algorithms for manipulating tree structures, symbol tables, etc., and machine-specific algorithms for expression tree modification, register allocation and generating machine instructions.

Several new ideas are presented in this thesis, including a novel method for generating machine instructions using tables, and a machine-invariant structure for generating variable length instructions for which the length depends on the distance between the instruction and its operand. Some standard ideas are also presented, for instance, the generation of code to control the execution of conditional statements, where emphasis is placed on their machine-invariant implementations.

This thesis should be of interest to two classes of people: those who are faced with the problem of implementing a high level language, possibly on a number of diverse machines, and those doing research into the problems of designing efficient algorithms for register allocation and code generation.

Before going further, some definitions need to be reviewed. An environment is a particular combination of machine hardware and software. The host environment of a program is the environment in which a program executes, while the target environment is the environment for which

the output of a program is intended. A program is said to be machine-dependent if it requires some set of host environment characteristics. A program is said to be machine-specific if it requires some set of target environment characteristics. The latter applies specifically to compilers, assemblers and loaders. A program which is not machine-specific is called machine-invariant. A program is said to be portable if it is less expensive to move it from one environment to another, than to implement the program separately in each environment.

Clearly, a compiler will be machine-specific because of differences in various target machine architectures, however dependence on the target environment can be limited to areas of the compiler which require some knowledge about the target machine. The code generation phases of a compiler are most heavily dependent on the target machine while the lexical scanning, syntactic analysis and global optimization phases can be entirely machine-invariant.

The remainder of this chapter briefly presents the Eh compiler as a whole. Chapter 2 discusses the structure of the load code input to Load, the Thoth Linking Loader (Sager, 1977), which relocates and links modules of load code. The output of Load consists of a core-image which, when loaded into the target machine's memory, can be



executed. Chapter 3 deals with code generation for named data structures represented by Eh external modules. Chapter 4 discusses code generation for executable Eh function modules.

### 1.1 Compiler Phases

The Eh compiler is divided into five functionally distinct phases: Lex, Parse, Clean, Data and Code. Figure 1.1 shows the structure of the compiler with respect to communication between phases. The communication is accomplished via files containing intermediate representations of the source input.

The first phase of the Eh compiler, Lex, reads the source input and produces two files, Strings and Lexil, which are used by subsequent phases of the compiler. Lexil contains an intermediate representation of the input in which manifests (a textual macro) have been expanded and keywords, operators, constants and identifiers have been separated into tokens.

The occurrence of an identifier in the source is represented in Lexil by a unique number, called an id. The text of an identifier is put in the Strings file and the id is the byte offset within Strings of the first byte of the identifier text.

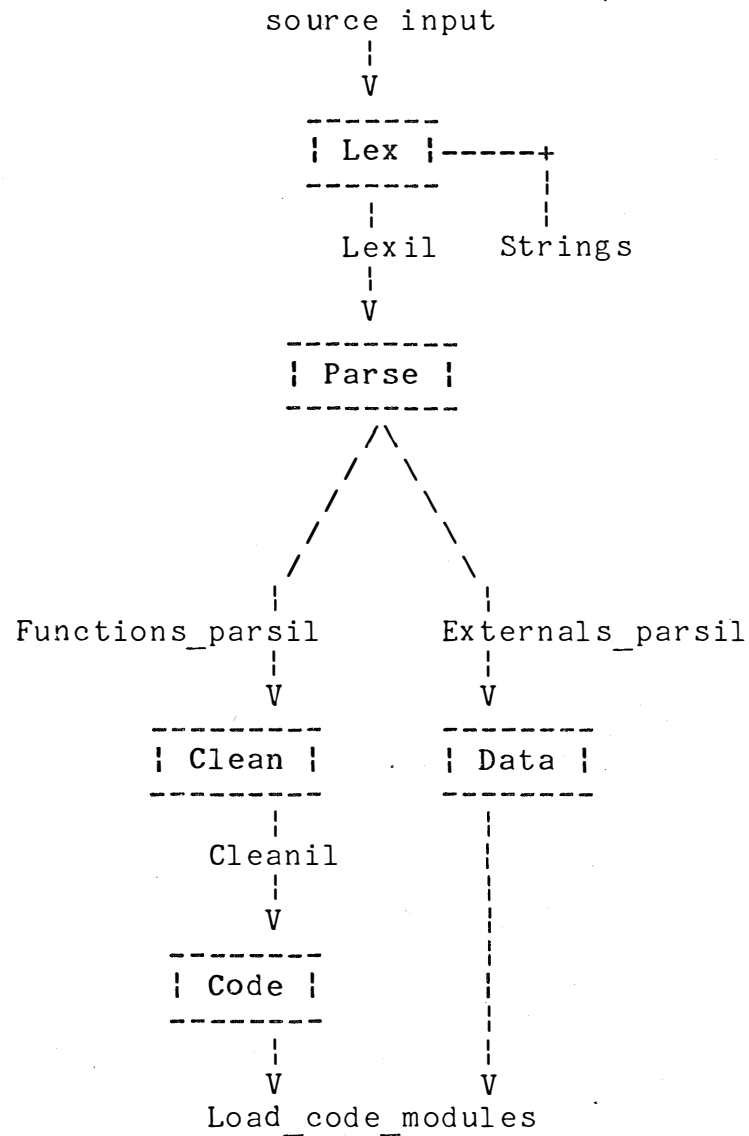


Figure 1.1 Compiler structure

Parse performs the syntactic analysis of the stream of tokens output by Lex. Parse's output consists of two files, Externals\_parsil and Functions\_parsil, which are intermediate representations of the external and function

modules found in Lexil. Expressions are represented in Functions\_parsil and Cleanil as trees in postorder form.

**Clean** performs control flow simplification and global machine-invariant optimizations on the function modules found in Functions\_parsil. The control flow simplifications include the removal of unreachable statements as well as the reduction of multiple tests and jumps to an equivalent test. The global optimizations include the removal of the passing of the number of arguments to functions which do not require it. Additionally, some expressions are modified to aid Code in the code generation effort. The form of the output, in the file Cleanil, is similar to that of Functions\_parsil. **Clean** also checks that the number of arguments passed to each function is within the limits specified in the function definition.

**Data** is responsible for generating load code for external modules defined in Externals\_parsil. Its output consists of a load code module for each external module. Information about the names of the modules, their sizes, relocation, initial values and references to other modules, is included in the load code modules. The load code modules are written to the file called Load code modules, which may later be read by either Load or the Library Editor (Sager, 1977). The Library Editor can be used to add load code

modules to a specified library. Additionally, it may be used to delete or replace modules or generate cross reference lists of modules in a library.

Code performs code generation for function modules output by Clean. The load code generated for each function module is also output to the file Load code modules. The load code contains machine instructions as well as information about the relocation of address fields, the sizes of modules and references to other modules.

## 1.2 Eh Language Domain

The Eh language is designed so that it can be implemented on most minicomputers. A computer model has been chosen based on characteristics of real minicomputers. Certain minimal assumptions are made about the target machine, including a minimum word length of 16 bits. The target machine integer representation and arithmetic are not specified. It may be 2's complement, 1's complement or even signed magnitude. At present, however, only 2's complement machines have been involved in the implementation of the Eh language. Currently, the Eh compiler makes an assumption about the host machine, namely, that the host word length is at least as large as the target word length. Compilation for target machines with larger word sizes would require

extensive changes to a number of data structures used in the compiler, in particular, those data structures which contain target machine constants.

### 1.3 Porting the Eh Compiler

To date, the Eh compiler has been ported using the half bootstrap method. A cross-compiler is built by supplying a set of 24 machine-specific functions and a set of manifests to be used along with the machine-invariant functions in **Data and Code**.

**Lex**, **Parse** and **Clean**, described by Stafford (1978), are machine-invariant. No changes in source code are required for any of these phases when a compiler for a new machine is implemented. In fact, they need not be recompiled to serve in a cross-compiler. The question of target machine arithmetic and integer representation is delayed until the code generation phases by using 128 bit precision 2's complement arithmetic in **Parse** and **Clean**.

**Load** and the **Library Editor** are similarly machine-invariant. To account for the machine-specific nature of relocation, word size etc., two special modules, **.Prolog** and **.Epilog** are input to **Load**. **.Prolog** is used to set up the relocation and target machine parameters, such as the number of bits per addressable word, and must be the

first module processed by Load. The last module, .Epilog, is used to describe the order in which various relocation areas will be loaded in the final core-image. Again, no additional source code or recompiling is required for Load or the Library Editor when porting the compiler. All phases of the compiler are machine-independant.

Once the cross-compiler is available, programs may then be compiled in the host environment and the resulting core-image modules transferred to the target environment by whatever means available, usually a communication line. The Eh compiler was originally implemented on a 36 bit machine and ported to 16 bit machines. Currently, the Eh compiler runs on a Honeywell 66/60 producing code for the Honeywell 66/60, the Texas Instruments 990, the Honeywell 6/30 and the Data General Nova. The compiler has been ported to run on the TI 990 where it produces code for the TI 990, the Nova and the Honeywell 6/30.

It is estimated that approximately 8 man years have been invested in implementing, redesigning and maintaining various aspects of the Eh compiler. Porting the compiler has required from 2 to 4 months depending on the complexity of the target machine and the experience of the people involved in the port. Most of that time was involved in designing the code bursts to be used for expression evaluation. The design

stage is the most time consuming because difficult decisions must be made about such things as register and temporary allocation, function calls and stack design.

To aid in the debugging effort, a set of diagnostics is available which test such things as the expression operators, function calling and select statements. The diagnostics have been designed to be machine-independent and machine-invariant and therefore do not adequately test for machine-specific problems such as running out of registers during expression evaluation, or twit statements. However they have proven to be a very useful tool for debugging a new compiler in a relatively short period of time. For instance, a recent port to the Honeywell 6/30 involved only 13 days of debugging. The diagnostics are also useful for testing changes in the compiler resulting from fixing bugs or modifying the language.

## Chapter 2

### Load Code Format

This chapter briefly describes the format of the load code modules produced by the code generation phases. It is included to provide an understanding of certain details of load code which are referenced in later parts of the thesis. For a complete discussion of modules and directives see Sager (1977).

Load code modules (hereafter referred to as "modules") consist of a series of directives to be executed by Load or manipulated by the Library Editor. Load executes the directives to produce an output file containing a core image which can be executed on the target machine. The Library Editor manipulates a small subset of the directives to build or modify a library of modules. A module is delimited by the start module **M** and end module **E** directives. The **M** directive serves to define the module name. External or global symbols are defined with the **G** directive and external symbol references are made with the **g** directive. Internal or local symbol references are made with the **t** directive and defined with the **T** directive.



## 2.1 Working Symbols

Two symbol tables are in use whenever a load code module is being processed by Load. The first contains information about all modules and their external symbol references. The second contains information about local symbols used within each module. To provide a shorthand of referencing these symbols, Load also maintains a working symbol dictionary of symbols used within the current module. The working symbol dictionary (Wsd) is an array of 256 words which may contain pointers to symbol table entries. Thus directives which involve references to symbols may use the working symbol dictionary index rather than the symbol name. The Wsd[0] entry always points to the entry for the current module and is set (and the remainder of the Wsd is cleared) when an M directive is encountered. The Wsd index of a symbol is required with the g and t directives. The Wsd is limited to 256 entries because only one byte is available in the load code directives for specifying the Wsd index.

## 2.2 Relocation Base Registers

Each symbol (local or global) defined in a module must have a relocation base register (Rbr) associated with it. An Rbr gives the base value for defining the value (usually an address) of each symbol in the module being processed. A

total of eight Rbrs (Rbr[0] through Rbr[7]) may be used. An Rbr may be used to relocate symbols to different areas in the target machine's memory, or it may be used to separate the core image into segments which reflect functionality, as would be required in machines with separate instruction and data spaces. The Rbr index must be supplied with a symbol in the M, G and T directives. An Rbr may be set to some value with the R directive, incremented with the I directive, and aligned (incremented to the next integer multiple of a specified value) by the A directive. The code generation phases of the Eh compiler treat Rbr[0] as a special Rbr which is set to zero and never incremented. This provides for symbols which represent absolute machine addresses.

### 2.3 Relocation Descriptors

**Load** allows up to eight relocation descriptors (Rd[0] through Rd[7]) to indicate how addresses are to be modified. Rd[0] is a special relocation descriptor which defines how Rbr values are modified. A relocation descriptor is defined with the D directive.

## 2.4 The Working Register

The working register is a 32-byte array used to hold data to be relocated. It is loaded with the L directive which contains up to 32 bytes of data, and is output as a core-image record with an O directive which immediately follows the L. The core image record consists of an absolute address followed by the data to be loaded at that address in the target machine.

The O directive outputs the core-image record after relocating fields within the working register. The O directive is composed of a byte pair for each field to be relocated in the working register. The byte pair consists of a byte offset in the working register, an Rd index and a Wsd index. The byte offset indicates the start of a field which is relocated using the specified relocation descriptor and working symbol.

## 2.5 Load Code Support Functions

The following is a description of the primary functions used by the Data and Code phases to generate the directives required by Load. The L directive is maintained in an array, the L-buffer, of 32 words, each containing one byte of data. The O directive is maintained in a 64 word array (the O-buffer) in which each pair of words corresponds to a

possible byte pair relocater.

`Load_word( data )`

Loads a target word of data into the L-buffer and updates the current location counter by the number of bytes in the word.

`Rload_word( Rd_index, Wsd_index, data )`

Loads a target word of data into the L-buffer, updates a location counter and constructs an O-buffer word pair using the `Rd_index`, `Wsd_index` and the L-buffer index at which the data was stored.

`Flush_load_bufs()`

Outputs the L and O directives using the contents of the L-buffer and O-buffer.

`Put_M_dir( Rbr, multiplier, module_name )`

Outputs an M directive for the specified module.

`Put_load_ref( 't' or 'g', Wsd_index, symbol_name )`

Outputs the t or g symbol reference directive for the specified symbol.

`Put_load_def( 'T' or 'G', Rbr, offset, name )`

Outputs the T or G symbol definition directive for the specified symbol.

`Put_I_dir( Rbr, increment )`

Outputs an I directive to increment the Rbr.

`Put_A_dir( Rbr, multiplier )`

Outputs an A directive to align the Rbr.

`Put_load_dir( directive )`

A general purpose function to output the vector specified by directive.

`Set_load_L_addr( Wsd_index, address )`

Loads the address in the first few bytes in the L-buffer and sets the first O-buffer word pair to relocate the address.

## 2.6 Load Code Example

Figure 2.1 is an example of a load code module. The module is similar to those produced by the TI 990 Data phase and results from the following source module:

```
External[]: 1, 2, 3, 4, .Get_str;
```

The example illustrates the order in which directives are output by the code generators. Line 1 defines the module External to be loaded relative to Rbr[2] and line 2 defines External to be an external symbol. The next line indicates a reference to the external symbol .Get\_str, which will be included in the code-image module produced by Load. This is followed by a reference to an internal symbol, X1. The L directives which follow are organized as an address field

(one word) followed by a byte count which is followed by that many data bytes (shown here as words rather than bytes). The corresponding O directives have been expanded for clarity. Each byte pair is split into three fields, the byte offset, the Rd index and the Wsd index. Thus, for instance, the first byte pair relocates the first few bytes in the L directive which corresponds to the address of the data stored in the remainder of the directive. Later, the T directive is generated which defines the internal symbol, X1, used in the module. At the end we find the Rbr increments and the end of module directive.

```

M 2 1 External
G 2 0000 External
g 250 .Get_str
t 1 X1
L 0000 2 0000
O ( 1 1 0) ( 4 1 1)
L 0000 10 0001 0002 0003 0004 0000
O ( 1 1 1) (12 1 250)
I 2 0002
T 2 0000 X1
I 2 000A
E 0 0 00 0

```

Figure 2.1 Example load code

## Chapter 3

### Code Generation For External Modules

Data is responsible for generating load code for external modules found in `Externals_parsil`. From the programmers point of view, the data defined in an external may contain integers or pointers. A pointer may point to other data segments which result from a vector or string within the current external module or to a completely separate module.

The generation of load code for an external module is a straightforward mapping of the data into a series of L and O directives. The load code may be simple, involving only the one target word of data, or it may be complex, involving a number of separate data segments. The latter case applies specifically to vector externals, consisting of a list of elements which are stored in consecutive memory locations. Since each element may be a pointer to another list of elements, the external cannot be built as one contiguous data segment. Instead, each list of elements is built using a separate data segment. Each such data segment is associated with a local symbol and a corresponding Wsd index.

### 3.1 Manifests

No source code changes are required for Data to be ported to a new machine, however certain manifests defining the target machine and relocation parameters are required and the source must be recompiled. These manifests are also used by Code and are found in a larger set of machine-specific manifests required by Code. The manifests used by Data are presented here.

WORD\_LENGTH - number of bits per Eh word on the target machine.

ADDR\_LENGTH - maximum number of bits required to represent a target machine address.

ADDR\_UNITS - number of bits per smallest addressable cell on the target machine.

EHADDR\_FACTOR - the factor,  $\text{WORD\_LENGTH} / \text{ADDR\_UNITS}$ , by which an Eh address is multiplied to obtain a machine address. It is used for converting from Eh addresses (word addresses) to machine addresses which may be, as in the case of the TI 990, byte addresses.

BYTE\_LENGTH - the number of bits per target byte.

LOAD\_ADDRLen - number of bytes required to hold a target machine address in the L directive, namely,  $(\text{ADDR\_LENGTH} + \text{BYTE\_LENGTH} - 1) / \text{BYTE\_LENGTH}$ . This is used for setting the address in the L-buffer corresponding



to the location in the core-image module at which the remainder of the L-buffer is to be stored.

MACHADDR\_FACTOR - the divisor used with byte oriented location counters to obtain an Eh address on the target machine: ADDR\_UNITS / BYTE\_LENGTH. One use for this manifest is for generating the Eh addresses of data segments which result from string or vector externals.

TARGET\_BPW - the number of bytes per target word, obtained by WORD\_LENGTH / BYTE\_LENGTH.

BYTE\_MASK - a mask to be used to obtain the low order byte of a target word.

EXTRN\_RB - the default Rbr to be used for loading an external module, that is, the contents of a simple external or the pointer used in string and vector externals.

DATA\_RB - the default Rbr to be used for loading the additional data segments used by both external and function modules.

PTR\_RELDISC - relocation descriptor to be used for relocating Eh pointers.

ADDR\_RELDISC - relocation descriptor to be used for relocating machine addresses. This is used primarily to modify the first LOAD\_ADDRLLEN bytes of the L directive.

ALIGN\_EXTRN - multiple to which the Rbr associated with an

external module must be aligned.

`ALIGN_EXTERNAL_DATA` - alignment for the Rbr associated with a data segment defined in an external module.

### 3.2 Data Segments

The `Externals_parsil` file contains definitions of three types of modules: simple externals, string externals and vector externals. Each external module in the input consists of a header and a body. The header is used in defining the existence of the module to Load while the body serves to initialize the module. Appendix A describes the format of external modules found in `Externals_parsil`.

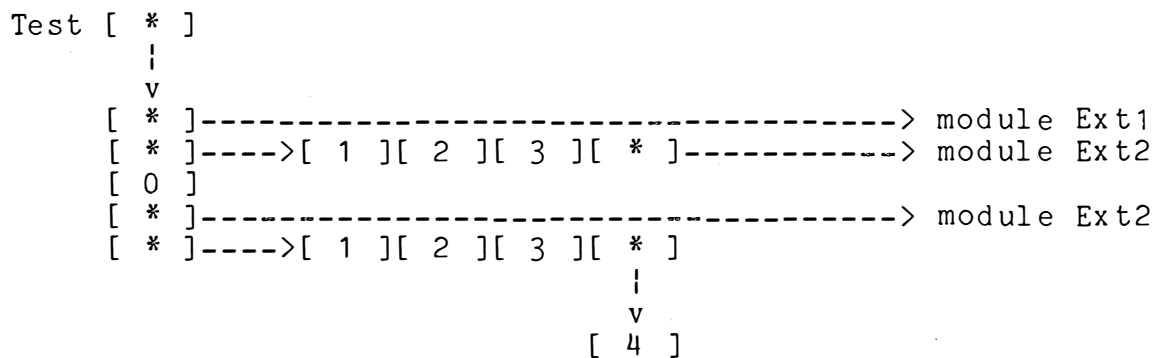
Simple externals are those which require only one word of storage. The content of the word is a constant or a pointer to another module.

String externals are those which require one word of storage which contains a pointer to a data segment which contains the text of a string.

Vector externals are similar to string externals in that they involve additional data segments. The initialization in each segment can consist of a combination of integers and pointers to other data segments, thereby resulting in a structure of nested data segments. For instance, the Eh statement

```
Test[]:Ext1, [ 1, 2, 3, Ext2 ], 0, Ext2, [ 1, 2, 3, [ 4 ]];
```

involves four data segments, illustrated by Figure 3.1. The structure involves a total of three levels.



\* means pointer

Figure 3.1 Example vector external

The first level includes four pointers, two of which involve second level data segments. Each of the second level data segments contains three constants and a pointer, one of which is a pointer to a third level data segment. The third level involves a one word data segment containing the single constant 4.

To economize on the number of data segments used in these instances, each level of nesting is associated with one data segment. The local symbol which identifies the data segment has a Wsd index equal to the level number associated with the data segment. Thus, the example in Figure 3.1

involves only three data segments, where the second data segment consists of two parts. Figure 3.2 illustrates the resulting use of data segments and the assignment of Wsd indicies to the local symbols which identify the data segments. Pointers are represented by a Wsd index and an offset within the data segment associated with the corresponding local symbol. Note that Wsd indicies for external symbols begin at 255 and are decremented for each additional unique external symbol reference. Local symbols defined within the module, however, are assigned Wsd indicies from 1 and up.

```

Wsd[0]=Test, Wsd[255]=Ext1, Wsd[254]=Ext2

Wsd[0]      Wsd[1]      Wsd[2]      Wsd[3]
[Wsd[1],0]->0[Wsd[255],0]
      1[Wsd[2],0 ]--->0[      1]
      2[      0]    1[      2]
      3[Wsd[254],0]  2[      3]
      4[Wsd[2],4 ]-+ 3[Wsd[254],0]
                    +->4[      1]
                        5[      2]
                        6[      3]
                        7[wsd[3],0 ]--->0[      4]

```

Figure 3.2 Data segment usage

### 3.3 Counter Stack

The initialization of a vector external effectively involves a stack of data segments. As each new level of the vector external is encountered, a new part of the corresponding data segment is used. The new part begins where the previous part left off and therefore has a starting address relative to the beginning of the data segment which depends on the previous use of the data segment.

To keep track of where one part ends and another part begins, a stack of counters is maintained. The counter stack consists of a vector of 256 words and a pointer. The pointer points to the entry in the vector which corresponds to the current level of vector external initialization. The content of the entry is a byte oriented location counter for the corresponding data segment. When a new level is encountered in the input, the pointer is advanced to the next entry in the counter stack. If this is the first time this entry is used, the location counter is set to zero and a load code `t` directive is generated, referencing a local symbol with `Wsd` index equal to the current level number. All initializations at the current level involve incrementing the current location counter by the size of the initialization (in bytes). When the end of the current level is encountered in

the input, the counter stack is popped by moving the location counter pointer back to the previous location counter in the stack.

### 3.4 Reference Table

The reference table contains the ids of all external symbols referenced in the current module along with the Wsd index assigned to each symbol. Data makes use of a separate hash table for maintaining such references. The least significant 6 bits of the id are used as an index into the hash table, each entry of which serves as the head of a linked list of symbol reference nodes which make up the reference table. If the id is not found in the list, a unique Wsd index is generated for the symbol, the index and id are added to the linked list, and a g directive is generated. The Wsd index which is returned by the search, may then be used in subsequent O directives for relocating references to the symbol. The table is used only for vector externals, since these are the only externals capable of containing references to more than one external symbol as part of their initialization.

### 3.5 Support Functions

Data is organized with respect to the types of external modules it can expect to find in its input. In addition to functions which process the three types of externals, various support functions exist for such things as extracting names from the Strings file and managing data structures. Some of the support functions required for generating code for vector externals are briefly discussed below:

`Push_loc` - called whenever a new level of initialization is encountered in the input, it moves the location counter pointer to the next entry in the counter stack corresponding to the next data segment. If this is the first time the level is encountered, a `t` directive is generated to reference the associated local symbol and the location counter is set to zero.

`Pop_loc` - the counterpart of `Push_loc`; it decrements the location counter pointer to that associated with the previous data segment.

`Pointer_gen` - generates a pointer to the data segment associated with the next level of the external vector.

The Main function and the three module processing functions follow.

```

Main( optc, argc, cmd_vec )
{
    extrn Module_name, Externals_rb, Error_out;
    auto c;

    .Put_str( "", Data" );
    .Flush();

    Initialize( optc, argc, cmd_vec );
    repeat
    {
        if( (c=.Get()) == 0 ) .Exit( 0 );
        if( c!=':' && c!='"' && c!='[' )
            Error( "invalid input in Main*n" );

        Clean(); \ initializations for each module

        Get_name( Get_id(), Module_name );
        Get_rbs();
        Put_M_dir( Externals_rb, ALIGN_EXTRN, Module_name );
        Put_load_def( 'G', Externals_rb, 0, Module_name );

        select( c ) \ process the module
        {
            case ':': Process_simple();
            case '"': Process_string();
            case '[': Process_vector();
        }
    }
}

```



```

Process_simple()
{
    extrn Externals_rb, Load_E_dir;
    auto c, wsd, id, name{32};

    if( (c=.Get()) CONSTANT ) Load_word( Get_constant(c) );
    else if( c=='g' )
    {
        Get_name( id=Get_id(), name );
        Put_load_ref( 'g', 255, name );
        Rload_word( PTR_RELDESC, 255, 0 );
    }
    else Error( "invalid character in simple external*n" );

    Flush_load_bufs();
    Put_I_dir( Externals_rb, EHADDR_FACTOR );
    Put_load_dir( Load_E_dir );
}

```

```

Process_string()
{
    extrn Data_rb, Last_data_ref, Counter;
    extrn Externals_rb, Load_E_dir;
    auto amt;

    Put_load_ref( 't', Last_data_ref=1, "X1" );
    Rload_word( PTR_RELDESC, 1, Counter[1]=0 );
    Copy_string( Get_constant( .Get() ), Get_id() );
    Flush_load_bufs();

    Put_I_dir( Externals_rb, EHADDR_FACTOR );
#ifdef ALIGN_EXTERNAL_DATA
    Put_A_dir( Data_rb, ALIGN_EXTERNAL_DATA );
#endif
    Put_load_def( 'T', Data_rb, 0, "X1" );
    if( amt = Counter[1]/MAHADDR_FACTOR )
        Put_I_dir( Data_rb, amt );
    Put_load_dir( Load_E_dir );
}

```

```

Process_vector()
{
    extrn Counter, Externals_rb, Data_rb;
    extrn Last_data_ref, Load_E_dir;
    auto c, wsd, id, min_length, name{32};

    min_length = TARGET_BPW * (Get_constant().Get()) + 1);

    Pointer_gen(); \ to first data segment
    Push_loc();

    repeat
        select( c = .Get() )
        {
            case CONSTANT: Load_word( Get_constant(c) );

            case 'g': \ external symbol reference
            {
                Get_name( id=Get_id(), name );
                wsd = Search_wsd( id, name );
                Rload_word( PTR_RELDESC, wsd, 0 );
                Flush_load_bufs();
            }

            case '"': \ string initialization
            {
                Pointer_gen(); \ to next data segment
                Copy_string( 0, Get_id() );
            }

            case '[': \ new level
            {
                Pointer_gen(); \ to next data segment
                Push_loc();
            }

            case ']': Pop_loc(); \ to previous segment

            default: break;
        }

    if( c )
        Error( "bad character in vector external*n" );
    *Counter = .Max( *Counter, min_length );
    Pop_loc();
    Flush_load_bufs();

    Put_I_dir( Externals_rb, EHADDR_FACTOR );
    for( wsd=1; wsd<=Last_data_ref; ++wsd )
    {
        ++Counter;
    }
#ifdef ALIGN_EXTERNAL_DATA

```

```

        Put_A_dir( Data_rb, ALIGN_EXTERNAL_DATA );
?end
        Put_load_def( 'T', Data_rb, 0, Create_name(wsd) );
        if( min_length = *Counter/MACHADDR_FACTOR )
            Put_I_dir( Data_rb, min_length );
        }
    Put_load_dir( Load_E_dir );
}

```

To continue with the example introduced in Section 3.1, Figure 3.3 shows the L and O directives for the data segments.

```

L 0000 2 0000
O ( 1 1 0 ) ( 4 3 1)

L 0000 2 0000
O ( 1 1 1 ) ( 4 3 255)

L 0002 2 0000
O ( 1 1 1 ) ( 4 3 2)

L 0000 8 0001 0002 0003 0000
O ( 1 1 2 ) (10 3 254)

L 0004 4 0000 0000
O ( 1 1 1 ) ( 6 3 254)

L 0008 2 0004
O ( 1 1 1 ) ( 4 3 2)

L 0008 6 0001 0002 0003
O ( 1 1 2 )

L 000E 2 0000
O ( 1 1 2 ) ( 4 3 3)

L 0000 2 0004
O ( 1 1 3)

```

Figure 3.3 Example Data output

The O directive byte pairs have been split into start

byte, Rd index and Wsd index for clarity. Relocation descriptors 1 and 3 correspond to machine address and pointers respectively. The load directives in Figure 3.2 are those which would result from the TI 990 Data phase, however, the Data phase for other 16-bit machines would differ only in the relocation descriptors.

### 3.6 Portability

Except for manifests, no source modifications are required when porting Data to a different target machine. The only machine specific parts of Data are those which make use of target byte size, word size and load code parameters which are all supplied by manifests. The manifests are used in the set of support functions used for generating load code directives and are defined in the machine specific manifest files required by Code. Thus, for all intents and purposes, Data is machine invariant since the specification of the required manifests for Code plus the manifests, `ALIGN_EXTERNAL_DATA` and `EXTRN_RB`, will result in a version of Data by simply recompiling the source code for Data.

Additionally, the source code for Data does not need to be changed in order to execute on a different host machine.

## Chapter 4

### Code Generation For Function Modules

Code is responsible for generating target machine code. Most of Code consists of 44 machine-invariant functions which read the input, build and modify expression trees, manage tables, and output load code directives. In porting the compiler, by far the greatest effort is spent on the problems of generating efficient machine code for various Eh statements and expression operators. The remainder of the source (24 functions and one set of manifests) must be specified for each machine-specific Code phase. These functions are machine-specific because they are directly involved with the target machine architecture (such as generating instructions) or they are involved with mechanisms which could make use of certain aspects of the target machine. An example would be the availability of fast table manipulating instructions which dictate a particular table organization thus preventing the machine-invariant part of Code from generating standard but possibly conflicting table organizations for select statements.

The process of generating code can be divided into four basic areas: expressions, control-path support, stack

support and select statement support. Generating code for expressions involves machine-specific modification of expression trees, register allocation and machine instruction generation. Control-path support involves the generation of instructions which depend on the existence of labels. A subset of the functions used in control-path support addresses the problem of generating instructions which depend on the distance between the instruction and its label operand. Stack support includes the assignment of addresses to elements in the stack frame, such as autos and temporaries, as well as the overall design and usage of the stack as a whole. Select statement support includes a number of machine-specific functions which generate code to perform various table lookup algorithms for the purpose of executing a select statement.

Like **Data**, **Code** uses data segments in which to place the generated code, however only three such data segments are used by the machine-invariant portions of **Code**. One data segment, hereafter referred to as the code segment, contains all of the machine instructions generated by the machine-specific part of **Code**. A second data segment is used to save the text of strings and a third is made available for saving select tables, constant pools, etc.

#### 4.1 Input And Overall Structure

The input to **Code** is found in the **Cleanil** file and is described by Stafford (1978). The information in **Cleanil** is structured so as to allow a finite-automaton to process function definitions in a machine-invariant manner, passing the machine-specific aspects on to functions which have been defined for such purposes.

**Cleanil** consists of statements which represent parts of a function module such as declarations, expressions, jumps, etc. Each statement consists of a flag byte identifying the kind of statement, followed by a sequence of bytes which make up the remainder of the statement. Appendix A describes the format of these statements.

**Funct\_gen** is a machine-invariant function which processes the function module represented by **Cleanil** statements. Its primary purpose is to coordinate the code generation in a machine-invariant manner. Hence, **Funct\_gen** is responsible for building the symbol table, managing data structures and building expression trees as well as calling on machine-specific functions to handle machine-specific tasks. These tasks include modifying the expression trees, allocating registers and generating machine code for expressions and selects. The following sections give an overview of **Funct\_gen** with respect to the input statements.

#### 4.1.1 Declaration Statements

A symbol table entry (or symbol) is created by Funct\_gen for all variables declared in the function module. Each entry is a vector with a minimum length of 5 words, as shown in Figure 4.1. The REF field contains a unique number generated by Parse which identifies the variable and is used in conjunction with the LINK field to locate the symbol in the symbol table.

| LINK                        | REF | VALUE | ADDRMOD | TYPE |
|-----------------------------|-----|-------|---------|------|
| 0                           | 1   | 2     | 3       | 4    |
| -----                       |     |       |         |      |
| STRING_ID                   |     |       |         |      |
| -----                       |     |       |         |      |
| 5                           |     |       |         |      |
|                             |     |       |         |      |
| NEXT_CONSTANT   CONST_VALUE |     |       |         |      |
| -----                       |     |       |         |      |
| 5                           |     |       | 6       |      |

Figure 4.1 Symbol table entry

Six bits of the REF field are used as an index into a 64 word hash table in which each entry is the head of a linked list of symbol nodes which index to that entry. Each new symbol node is added to the end of the appropriate linked list. A symbol is added to the table for auto variables, external variables, labels, constants and strings.



String symbols include an extra field `STRING_ID` which saves the ID of the string text for machine-specific use. The machine-invariant function `Process_string` is called by `Funct_gen` to generate load code directives which define the text of the string. Constant symbols include two extra fields; `NEXT_CONSTANT` and `CONST_VALUE`. `NEXT_CONSTANT` is a link field used to create a pool of constants. The machine-invariant function `Constant` may be used to retrieve a constant symbol or to create a constant symbol and link it into the pool. A symbol such as the one created by `Constant` is called a pseudo symbol and differs from those found in the symbol table by the fact that the `REF` and `LINK` field are zero and cannot be used to locate the symbol in the table. Hence, the requirement for a separate link field. The `CONST_VALUE` field of a constant symbol is used to hold the actual value of the constant.

The use of the `VALUE` field of a symbol varies with the kind of symbol. For instance, label symbols have a `VALUE` which is the value of the location counter at the time the label symbol is defined, whereas auto symbols have a `VALUE` which is set by the machine-specific function `Next_auto`. The value usually corresponds to an offset from a target machine stack pointer or register.

The `ADDRMOD` field is used to hold a target machine

address modification. An example is an ADDRMOD which is the bit pattern which indicates the use of a stack pointer or register for auto variables. The ADDRMOD field, along with the VALUE, would be included with target machine instructions which make use of the auto symbol. The TYPE field is actually a composite field as outlined in Figure 4.2.

| D     | TYPE | REL | WSD   |
|-------|------|-----|-------|
| 15 14 |      | 10  | 07 00 |

D = 1 if symbol defined, 0 otherwise  
referred to by SYMBOL\_DEFINED

TYPE = variable type, label, auto, constant, etc

REL = Relocation descriptor number used to  
relocate this symbol

WSD = Wsd index associated with this symbol,  
i.e. data segment, code segment or  
external module reference

Figure 4.2 Symbol type descriptor

Both the TYPE and ADDRMOD fields are set with the aid of manifests (see Section 4.2) when the symbol is first entered into the symbol table.

As indicated previously, new symbols are added to the ends of their respective linked lists. Adding a new symbol to the end of a linked list is not as inefficient as it may

seem at first, primarily because the average number of symbols used in a function module is well below the number of entries in the hash table. For example, in the 154 function modules of the Lex phase, the Honeywell 6/30 Code phase and the Eh command combined, there were 3401 symbols referenced, for an average of 22.1 symbols per module. Of these modules, only 7 resulted in symbol tables containing linked lists of more than one symbol. The average number of comparisons used to search for symbols in the 7 modules is listed in Figure 4.3. The average number of comparisons for all 154 modules is approximately 1.059. The low average is due also to the order in which symbols are defined in a function module. All non-label symbols are entered in the table before any label symbols because they are declared at the beginning of the function module. This has the effect of ensuring that non-label symbols are as close as possible to the front of their respective linked lists. This is important in view of the fact that non-label symbols are referenced much more frequently than labels symbols.

| <u>module name</u>   | <u>symbols</u> | <u>references</u> | <u>comparisons</u> | <u>average</u> |
|----------------------|----------------|-------------------|--------------------|----------------|
| Funct_gen            | 107            | 547               | 702                | 1.283          |
| Register_constraints | 63             | 276               | 336                | 1.277          |
| Result_loc           | 97             | 685               | 786                | 1.147          |
| Printree             | 66             | 220               | 226                | 1.027          |
| Lex                  | 88             | 371               | 485                | 1.308          |
| Search               | 59             | 312               | 339                | 1.086          |
| Main                 | 169            | 690               | 1122               | 1.626          |

Figure 4.3 Average comparisons per module

Where non-label symbols tend to be referenced many times in expressions, label symbols tend to be referenced twice; once when the label is defined and once for when it is referenced in a jump. The exception to this is the select statement, in which each case references a label following the select statement.

Select statements also increase the number of label symbols because there is a unique label associated with each case in the select statement. All of the modules in Figure 4.3 contain large select statements and this is the main reason for the size of the respective symbol tables.

#### 4.1.2 Expression Statements

Expression statements are generated by Parse and processed by Clean for every expression in the original source. These include expressions found in conditionals,

returns and selects. The flag byte for the statement indicates the kind of expression involved. With the exception of the return statement, the machine-specific functions `Modify` and `Allocate_registers` are always called to manipulate expression trees built from the input by the machine-invariant function `Build_tree`. Expressions are presented in Polish Postfix form after being first modified by both `Parse` and `Clean`. The modification performed by `Parse` involves reversing the operands to all assignments, function calls, byte and word subscripting. This ensures that a strict left to right walk of the expression tree will result in the correct order of expression evaluation. `Clean` further modifies binary operators which have a constant as the left operand by reversing the operands and indicating the change if the operator is non-commutative. This ensures that if there is a constant involved, it will always appear as the right operand, thus simplifying the amount of checking which may be required by the machine-specific portions of `Code`. `Clean` also performs a Sethi-Ullman type balancing operation on expression trees, which is described in detail by Stafford (1978).

Expression trees consist of operator nodes and symbol nodes where symbol nodes are the leaf nodes and operator nodes are internal nodes. Operator nodes are always 5-word

vectors, as shown in Figure 4.4. LEFT and RIGHT contain pointers to the left subtree and right subtree when such subtrees exist. RESULTIN and SCRATCH are two fields made available for use by machine-specific functions. The OP field contains the operator code for a corresponding Eh operator or a machine-specific special operator. Note that the OP field and the REF field have the same offset within their respective nodes. Since operation codes are always odd and reference numbers are always even, the rightmost bit of the fields can be used to indicate the type of node (operator or symbol).

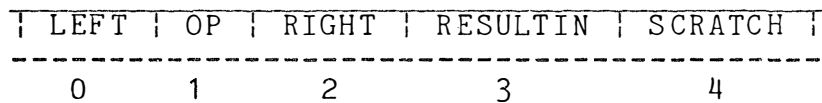


Figure 4.4 Expression tree node

For the return statement, Modify and Allocate\_registers are called only if there is an expression, since the expression may be null. Similarly, the machine-specific function Arith\_expr\_gen is called only if the return expression is not null. In either case the machine-specific function Return\_gen is then called to generate the code required to effect a return from the function.

For conditional expressions, the machine-invariant

function Logical `expr_gen` is called to generate code using the machine-specific function `Relation_gen`. All other expression statements result in `Funct_gen` calling `Arith_expr_gen` to generate the code to evaluate the expression. For select statements, the machine-specific function `Jump_gen` is also called to generate a jump to the code which will determine which case code to execute based on the selected value.

#### 4.1.3 Select Statements

Regardless of the kind of `.select` (string select or word select), there are three parts associated with each select statement.

The first part, indicating the beginning of a select, involves an expression which is to be evaluated. After the expression is evaluated, a jump is generated which references a label which is not defined until the third part of the select is encountered. The jump serves to bypass the code which is generated by the second part of the select, namely, the set of select cases and the `Eh` statements associated with each case.

The second part of a select statement involves the select cases. Certain minimal information must be kept as the select cases are being processed. This includes the

label at which a particular select case is located and the corresponding case value (or values). The information is maintained in a data structure which for word selects also includes the upper and lower bounds of values over which the select statement operates and the total number of cases. For string selects, the machine-invariant function `Process_string` is called to output the load code directives defining the text of the string. Because a select statement can occur within the case of another select, the data structures are linked in a stack so that when the processing of the current select is finished, the data structure for the encompassing select can be made available.

The third part of a select consists of code (selection code) to be generated which will use the result of the expression evaluated in the first part to determine which of the select cases to execute. For word selects the machine-invariant function `Wdselect_gen` is used to call one of several machine-specific functions which generate the actual selection code (see Section 4.5). For string selects, the machine-specific function `Str_select` is called to generate the selection code.



#### 4.1.4 Miscellaneous Statements

A prolog statement follows the last auto declaration corresponding to a formal argument of the function and precedes any other auto declarations. Its purpose is to mark the point in the input after which code may be generated. To this end, certain information such as the Rbr numbers to be used for code and data segments is supplied in the body of the prolog statement. Load code directives are generated by `Funct_gen` to define the function module and its entry point. Load code directives are also generated which reference some local symbols. The local symbols are associated with data segments used for addressing certain parts of the function stack (see Section 4.4). `Funct_gen` calls the machine-specific function `Prolog_gen` to perform certain operations pertaining to machine code to be generated for entering a function.

An end-of-declaration statement signifies that there are no more non-label symbols to be declared.

It is possible for certain variables (such as `Thoth` system variables) to be highly machine specific in their implementation. In fact one such variable, `.Active`, seems best used as an external on the TI990, an absolute memory location on the Nova and a hardware register on the Honeywell 6/30. To allow the different implementations of

.Active on various machines, the keyword `active` has been added to the language and its use is presented to the `Code` phase by the use of a special-keyword declaration. The declaration statement is encountered just prior to the first use of the keyword. A symbol table entry is made for the keyword, however no fields are initialized. The symbol is processed by the machine-specific function `Process_keyword` which is responsible for setting the fields of the symbol node to reflect the way in which the symbol is used. Thus, on the TI990, the symbol is initialized to appear as an external and on the Nova it is initialized to appear as an absolute memory location.

A jump statement indicates that code is to be generated which causes a jump to the label specified in the jump statement. The machine-specific function `Jump_gen` is called to generate the code whenever a jump statement is encountered.

A label statement is encountered whenever a label is to be defined. The machine-invariant function `Backplug_gen` is used to define the label and generate any instructions (using the machine-specific function `Code` described in Section 4.3) which could not be generated because the label had not been defined (see Section 4.7).

A `twit` statement consists of an expression which

resembles a call to an Eh function. `Funct_gen` removes the function call operator node and calls the machine-specific function `Twit_gen` to process the resulting vector of pointers to symbols corresponding to the arguments.

The machine-specific function `Disable_gen` is called to generate code to effect disabling of interrupts whenever a `disable` statement is encountered. Similarly, the machine-specific function `Enable_gen` is called to generate code to effect enabling of interrupts when an `enable` statement is encountered.

The end-of-module statement signifies the end of the current function module definition. The machine-specific function `Epilog_gen` is called to perform actions concerned with the end of a module definition. These include generating load code directives defining the local symbols used for relocating stack references (see Section 4.4).

## 4.2 Manifests

The portability of Code is largely due to the use of manifests throughout the source. By changing the definition of a manifest, effectively every function which makes use of that manifest is changed. The relative ease with which a manifest definition is changed, as compared to changing the functions, contributes greatly to the overall ease with

which a compiler may be ported.

The following manifests used by the machine-invariant Code functions pertain specifically to the target machine.

PLUS\_INFINITY - largest positive integer value on the target machine as a 2's complement integer.

MINUS\_INFINITY - largest negative integer value on the target machine as a 2's complement integer.

AUTO\_ADDRMOD - the address modification required for addressing auto variables. This and other ADDRMOD manifests may be the actual bit pattern of the address modification as would be found in the target instructions using the address, or they may be unique numbers recognizable as such by the Code function.

LABEL\_ADDRMOD - the address modification required for addressing labels (both those generated internally by the compiler and those existing in the source).

EXTRN\_ADDRMOD - the address modification required for addressing externals or functions.

STRING\_ADDRMOD - the address modification required for addressing literal strings used in the function module.

CONST\_ADDRMOD - the address modification required for addressing constants used in a function module.

AUTOS\_ARE\_DONE - an Eh statement (possibly null) to be executed after the last auto declaration is processed

by `Funct_gen`. This may be useful for some implementations which, for instance, require information about how many stack locations are reserved for autos, before the first expression is processed.

`MS_CLEAN` - an Eh statement (possibly null) executed by the machine-invariant function `Clean`, to take care of machine-specific initializations at the beginning of each module.

`MS_INITIN` - an Eh statement (possible null) executed by the machine-invariant function `Initialize`, to take care of machine-specific initializations at the beginning of Code execution.

`JUMPTABLE_LIM` - the maximum number of non-range cases in a select to be tolerated using jump tables before it is more efficient to simulate if statements. This number is a reflection on the space requirements of both the table space and code space involved in using jump tables.

`STRAIGHTTEST_LIM` - the maximum number of non-range cases to be tolerated in simulating if-else statements before it is more efficient to perform a binary search. This number reflects the code space requirements involved in simulating if-else statements (see Section 4.5).

`ABSOLUTE_RB` - Rbr to be used for loading absolute data. This

is used by Code for relocating the data segments representing portions of the stack frame of a function module.

STRINGS\_RB - Rbr to be used for loading the data segment containing the text of literal strings used in function modules.

CODE\_RB - Rbr to be used for loading the code segments containing machine instructions generated for function modules.

STRING\_RELDISC - relocation descriptor to be used for relocating pointers to literal strings used in function modules.

AUTO\_RELDISC - relocation descriptor used to relocate pointers to auto variables in function modules.

ALIGN\_FUNC - Eh address multiple to which the Rbr associated with a code segment must be aligned.

ALIGN\_FUNCTION\_DATA - similar to ALIGN\_EXTERNAL\_DATA but pertaining to data segments defined by function modules.

### 4.3 The Code Function

The generation of machine instructions is accomplished on many levels. The lowest level involves the Load\_word and Rload\_word functions which are described in Chapter 2. The

next level involves a function called `Code` which sets up arguments for and calls `Load_word` or `Rload_word` to output one complete machine instruction. The arguments to `Code` closely resemble the format one would expect when coding in assembly language, thus, the `Code` function is machine-specific. The next higher level involves functions such as `Arith_expr_gen` which generate more than one machine instruction using multiple calls to the `Code` function. The highest level is `Funct_gen` which, among other things, is responsible for calling `Arith_expr_gen`, `Wdselect_gen`, etc.

The `Code` function is responsible for generating one machine instruction. It is called with the machine opcode and up to the maximum number of operands possible in a machine instruction. The `Code` operands are symbol table entries, such as auto variables, or specially created pseudo symbol table entries such as absolute address operands. The number of target machine words which result depends on the machine opcode and its required operands and may involve multiple calls to `Load_word` or `Rload_word`. The `Code` function is implemented as a select statement in which an opcode or a group of opcodes is represented by a select case. The use of the `Code` function aids in the translation of higher level operations into machine instructions because it allows the use of manifests and operands which resemble a machine

instruction or its assembly language equivalent. For instance, a straightforward coding of the expression,  $A = B \mid 7$ , using the TI 990 Code function would be:

```
Code( .LI., R12, Constant(7) );    \ load register 12 with 7
Code( .SOC., R12, B );              \ OR the contents of B
                                   \ into register 12
Code( .MOV., A, R12 );              \ move result in register
                                   \ 12 to A
```

The Code function is also used for generating twit statements. The machine-specific function Twit\_gen is used to perform checks on the twit statement arguments and to provide the correct arguments to the Code function. For the TI 990, the following three twit statements would result in roughly the same three calls to the Code function as in the previous example.

```
twit( .LI., "r12", 7 );
twit( .SOC., "r12", B );
twit( .MOV., A, "r12" );
```

Twit\_gen first translates the string "r12" into a pseudo symbol table node which corresponds to register 12, and the value 7 is used to create a symbol table node corresponding to the constant 7. Appendix B contains a listing of the TI 990 Code function.



#### 4.4 Stack Support

Although there are no specific stack designs imposed by Code, a fixed size stack frame implementation is preferable. Variable size stack frames result when the number of used stack locations vary with respect to the number of temporary locations in use at the time of a function call. A fixed size stack frame means that the same number of stack locations are in use throughout the function execution and therefore stack space must include the maximum number of temporaries used by a function. While this may at times result in certain stack temporaries not being in use at the time of a function call, there are distinct advantages in using a fixed size stack frame:

- 1 The value by which the stack pointer must be incremented does not need to be specified with each function call, thereby moving the stack increment specification from the caller to the called function. This has the effect of saving code space as well as execution time since the increment is processed once only.
- 2 Stack overflow can be detected more cheaply than with variable size frames because only one value (the frame size used by a function) is involved, rather than a frame size and a stack pointer increment supplied by

the calling function.

- 3 The upper bound on the total stack size required by a specific chain of function calls can be calculated with less effort because only one frame size for each function is involved.

A more subtle advantage concerns the passing of the argument count (called Nargs), to called functions which require it. With a fixed size frame, the Nargs location on the stack never changes, and Nargs needs to be set only if it differs from the previous value. With variable sized frames, Nargs needs to be set for each function call requiring it, since the Nargs location may vary or be reused for temporaries in the course of the function execution. As an example of the savings incurred in code space, consider the compilation of the Thoth operating system for the TI 990. In all, there are 351 places in the resulting code in which Nargs needs to be calculated and passed to the called function. Of these, 90 calculations can be accomplished using increment or decrement instructions, requiring two words of code as opposed to three used to move a constant to the Nargs location. Another 54 calculations can be ignored completely because the required Nargs value is the same as in the previous use of Nargs. This results in a saving of 252 words of code space, or approximately 24% of the total

code space (1053 words) which would be needed to pass Nargs with a variable sized stack frame. The stack design for the TI 990 is discussed in Appendix C.

The execution of a function usually relies on various actions at the start of execution, in particular, those concerning the stack frame. These include modifying the stack frame pointer to point to the stack frame associated with the invoking of the function, as well as checking for stack overflow conditions. In most cases this is accomplished by calling an assembly coded subroutine. The machine-specific function Prolog\_gen is responsible for generating code to call the subroutine or to perform the stack operations.

Prolog\_gen( nargs ) \ for the Nova

```

\ Determine which function prolog subroutine to call
\ based on whether the function contains an unbounded
\ number of arguments (ie. questionable).
\ Generate the code to call the subroutine.
\ Save the code segment location at which to stuff the
\ stack increment via Epilog_gen.
{
  extrn Counter, Framesizelocn, Prolog_rtn, Ac2, Ac3;

  Put_load_ref( 'g', Wsd_index(), ".Fcn_prolog" );
  if( nargs == 0377 ) \ questionable function
    VALUE[Prolog_rtn] = .QFCN_PROLOG;
  else
    VALUE[Prolog_rtn] = .FCN_PROLOG;
  Code( .STA., Ac3, Ac2 ); \ save return address
  Code( .JSR., Prolog_rtn ); \ increment and check stack
  Framesizelocn = *Counter;
  Load_word( 0 ); \ gets plugged by Epilog_gen
}
```

It is often more convenient, if not necessary, to refer to the stack by its constituent parts (autos, auto vector space and temporaries) rather than as a whole. The necessity often grows out of a particularly efficient stack design which normally could not be used unless the size of each part is known before any code is generated. Such is the case with stack design for the Nova, in which the stack pointer splits the stack frame into two parts: autos, vector space and temporaries on one side of the pointer and arguments passed to called functions on the other side. The stack increment in this case does not need to include the space used by the function for passing arguments to called functions. Consequently, there is no need to pass an increment to the called function in order to bypass the calling functions autos and temporaries. The problem, however, is that the caller's temporary stack space lies between the stack frame pointer and its autos. Since it is not known how large the temporary stack space will be until the end of the function definition, the address of an auto cannot be calculated as an absolute offset from the stack frame pointer. For this reason, each part of the stack frame is relocated so that references to individual elements in the stack frame may be made relative to a data segment which represents each part. The local symbols `..auto`, `..vec` and

..temp are used to identify the data segments and the external variables Auto\_top, Autovec\_top and Temp\_top are supplied for keeping track of the sizes of the respective parts. References to these local symbols are generated just before Prolog\_gen is called by Funct\_gen. A more detailed description of the Nova stack design may be found in Appendix D.

The machine-specific function Epilog\_gen is used to perform whatever actions are necessary to end a function module definition. At the very least, it is used to specify the relocation of autos, auto vectors and temporaries within the stack frame by generating the load code T directives which define the local symbols associated with the stack parts. The Wsd indicies are specified by the machine-invariant manifests WSD\_AUTO, WSD\_AUTOVEC and WSD\_TEMP respectively.

Epilog\_gen() \ for the Nova

```

\ Dump the remainder of the constant/address pool
\ Define local symbols "..auto", "..vec", and "..temp".
\ Backplug the framesize location used in the prolog.
{
  extrn Autovec_top, Nargs_passed, Auto_top, Framesizelocn;
  extrn Temp_top, Counter;
  auto  next_frame, frame_size;

  Dump_pool();

  frame_size = Auto_top + Autovec_top + Temp_top + 2;
  if( Nargs_passed ) ++frame_size;
  next_frame = frame_size;
  Flush_load_bufs();
  Set_load_L_addr( WSD_CODE, Framesizelocn );
  Load_word( -next_frame ); \ stuff stack increment
  *Counter -= 2; \ to set proper size of module;

  Put_load_def( 'T', ABSOLUTE_RB, --next_frame, "..auto" );
  next_frame -= Auto_top+Autovec_top;
  Put_load_def( 'T', ABSOLUTE_RB, next_frame, "..temp" );
  Put_load_def( 'T', ABSOLUTE_RB, ++next_frame, "..vec" );

  Set_frame_size( frame_size ); \ save for stack bounding
}

```

#### 4.5 Select Statement Support

Funct\_gen calls the machine-invariant function Wdselect\_gen at the end of each word select statement. Wdselect\_gen decides which type of select code is most efficient given the number of cases involved and the relative density of the values of the cases. In addition to simulated if-else statements, three types of select code may result which use tables described below. Although the description of the tables seems to imply a particular

organization, the actual organization can vary with each Code phase.

Binary search tables are two-word per entry tables in which the first word contains the value of a case and the second word contains the address of the code to handle the case. In most cases the table is split up into two subtables which are accessed in parallel. The table is that which results from performing a heap sort on the case values and the standard algorithms, for example Knuth (1973), are used for comparing the selected value with the case value and jumping to the label corresponding to the case code. Binarysearch\_gen is the machine-specific function which is called to generate the code and build the required tables.

Jump tables are one-word per entry tables which contain the address of the case code to be executed. The index of an address in the table is the index of the corresponding case value relative to the minimum value of all cases. Table entries which have no corresponding case value (default entries) contain the address of the default case code if a default case exists. Otherwise they contain the address of the code following the selection code. Both simple integer cases and range cases (resulting in one entry per value in the range) are included in the jump table.

This method is inefficient in space if the number of

default entries exceeds the number of non-default entries. Table space for twice the number of non-default cases is all that is required to implement a binary search table. The machine-specific function `Jumptable_gen` is called to build the table and generate the required code. Of all the select codes, jump tables are the fastest in terms of execution time, however, `Wdselect_gen` will not choose jump tables if the space exceeds that used by other select codes.

Pair tables consist of three entries. Two of the entries contain the upper and lower bound of a case range. The address of the case code associated with the range is contained in the third entry.

A common approach to building pair tables takes into account the fact that overlapping cases are not allowed in selects. The table is first sorted by lower (or upper) bound case values and then split into two tables which are accessed in parallel. The first table contains a sorted list of alternating lower and upper bound case values in which the lower bound values have been decremented by one. By using a single less-than-or-equal test on each bound value, the decision to jump to case label or default label can be made depending on the index at which the less-than-or-equal test became true.

For instance, if the test is true on an even index in



the table, then it was true in the comparison against the upper bound value and thus falls within the original range specified in the select. If the test is true on an odd index, then it was true on the comparison against the lower bound minus one, that is, outside any range specified in the select. In the case of resulting in true on an even index, the index may then be halved to produce an index into the second table containing the address of the case code. Otherwise, the default label is chosen. In the situation where the lowest lower bound is minus infinity, the decisions regarding the index are simply reversed and the tests are started at the lowest upper bound.

This particular implementation of pair tables is efficient in terms of execution since it involves only one type of test within a small loop, plus one extra test when the loop is exited. The machine-specific function `Pairtest_gen` is called to build the tables and generate the selection code.

A fourth type of select code may result which does not use tables. The machine-specific function `Straighttest_gen` is used to generate code to simulate a series of if-else type statements wherein each case value is tested and if it matches the selected value, a jump is made to the corresponding case code. On most machines the if-else

simulation is more space efficient when a small number of cases (10 to 15 cases) are involved.

Wdselect determines which type of select code should be generated by using the properties of the different table types and machine-specific limits. The choice is based on table space efficiency only and uses the limits to aid in determining the code space efficiency of the various select codes which may result. Once a decision is made, Wdselect calls the corresponding machine-specific function to build the table and generate the code. Table building is left to the machine-specific functions so that advantage may be taken of any table type machine operations.

```

Wdselect_gen()
{
    \ Choose which code to generate for word select.
    extrn Select_ptr;
    auto min, max, ncases, size;

    ncases = NUMBER_OF_CASES[Select_ptr];
    min = MIN_CASE[Select_ptr];
    max = MAX_CASE[Select_ptr];
    size = max-min;

    if( min >= MINUS_INFINITY/EHADDR_FACTOR &&
        max <= PLUS_INFINITY/EHADDR_FACTOR &&
        size >= 0 && size < 2*ncases )
    {
        if( ncases >= JUMPTABLE_LIM ||
            INTERVAL_CASE[Select_ptr] )
            Jumpable_gen();
        else
            Straighttest_gen();
    }
    else if( INTERVAL_CASE[Select_ptr] ) Pairtest_gen();
    else if( ncases < STRAIGHTTEST_LIM ) Straighttest_gen();
    else Binarysearch_gen();
}

```

## 4.6 Expressions

Each expression encountered in Cleanil goes through three stages: modification, register allocation and code generation. Expression trees are built so that a left to right traversal of the tree represents the optimal execution order as determined by Clean.

### 4.6.1 Expression Tree Modification

The optimal execution order determined by Clean may not always be possible or efficient for some particular machine.

A typical modification involves the removal of the `IL_VALUE_RQD` operator from the root of an expression subtree. `IL_VALUE_RQD` is inserted in expression trees by `Clean` in areas which could be affected by side effects resulting from function calls, assignments, or increments and decrements. In such cases, the result of a subexpression must be safely stored in a temporary. Unfortunately, `Clean` cannot know which operators normally leave their results in temporary registers or stack locations, thus making the `IL_VALUE_RQD` operator superfluous in such situations.

Two machine-invariant modification functions are available for modifying expressions involving function calls. `Modify_args` is used to insert an assignment operator for each argument in a function call. The assignment is made to a symbol node returned by the machine-specific function `Assign_arg` which is passed the argument number. `Modify_args` also modifies the function call operator node (`IL_FCTN_CALL` or `IL_NFCTN_CALL`) so that the number of arguments passed to the called function can be retained for the code generation. Figure 4.5 illustrates the modification for the source statement, `Function_B( argument_2 );`.

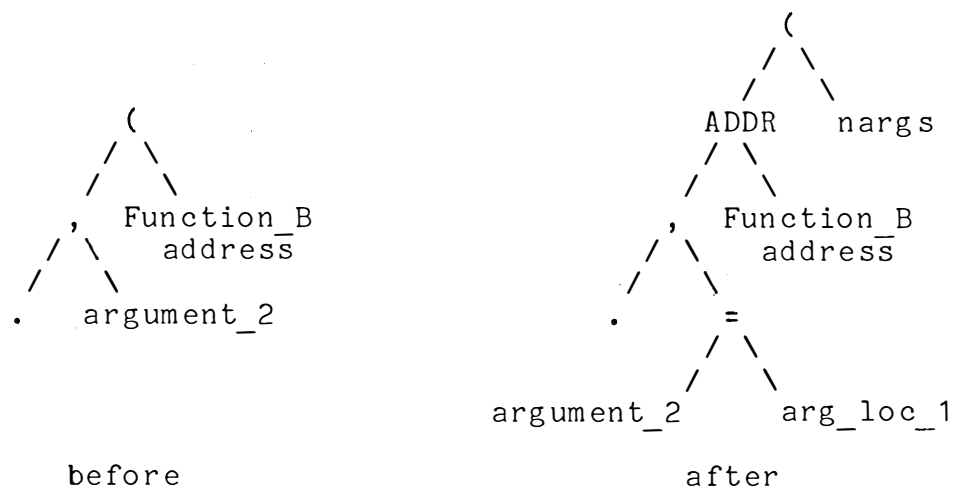


Figure 4.5 Modification of function arguments

Modify\_calls is used to ensure that any value being passed as an argument to a function is safely stored in a temporary if another function call must be made to evaluate a later argument. The safe store requirement is a consequence of the Eh language having a well defined order of evaluation which includes side-effects. For example, consider the expression:

```
Function_A( argument_1, Function_B(argument_2) );
```

Since arguments must be evaluated from left to right, the result of argument\_1 will be stored in the first argument location on the stack before Function\_B is called to supply the second argument result. With fixed size stack frames, there is only one set of locations at which all arguments are stored. This means that argument\_1 cannot be moved to

the first argument location until Function\_B has finished using the first argument location for its own argument\_2. Modify\_calls replaces the assignment to an argument location with an assignment to a temporary symbol returned by the machine-invariant function Alloc\_temp. An assignment from temporary to argument location is then inserted at a place in the function call subtree at which it is safe to set the argument location. The modification is not made when the nesting of function calls occurs on the first arguments only. Additionally, if the argument is a constant, no safe store is generated. Instead, the assignment of constant to argument location is made at the aforementioned safe node in the function call subtree.

The use of Modify\_calls is optional for two reasons. First, Modify\_calls is used only for fixed size stack frame implementations. Second, in the situation where an Eh operator does not have its counterpart in a target machine instruction, the operator is sometimes evaluated by calling an assembler subroutine to perform the operation using other available instructions. Since the machine-invariant part of Code does not know about such deficiencies, Modify\_calls cannot arbitrarily be called since any operator may, by way of subroutine, use the locations in which an argument has been stored. Figure 4.6 and Figure 4.7 show the effect of

the modifications for the above example expression. The expression subtree for the call to Function\_B is the same as that in Figure 4.5 and is simply indicated by the leaf named F\_B.

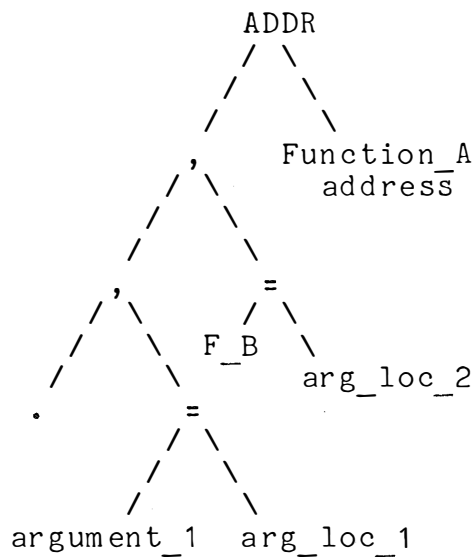


Figure 4.6 Nested function calls

Modify\_calls also replaces IL\_FCTN\_ARG with IL\_NULL since IL\_FCTN\_ARG has effectively become a place holder and therefore becomes a null operation.

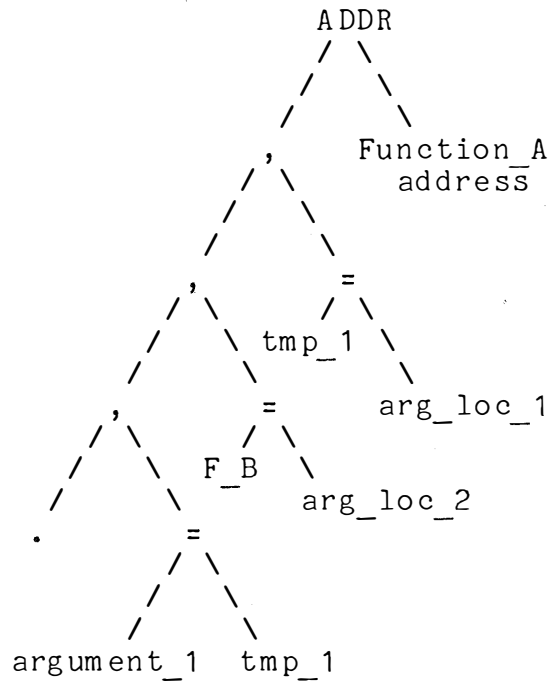


Figure 4.7 Modification of function calls

Modification of expression trees is initiated by a call to the machine-specific function `Modify`. In addition to the root of the expression tree, the flag byte of the `Cleanil` statement is also passed to `Modify`. This facilitates specific modification of expressions with respect to return or select statements so that the result of an expression can occur in a place which the corresponding operation (return or select) can use consistently. For instance, the TI 990 Code phase ensures that the result of a return or select expression occurs in register 12 by adding a special register assignment to the root of the expression tree.



#### 4.6.2 Register Allocation

Register allocation is performed by the machine-specific function `Allocate_registers`. Allocation of registers is essentially machine-specific because the number and usage of machine registers vary from machine to machine. For example, the Nova has 4 registers, only two of which may be used for indexing; the TI 990 has 16; and the Honeywell 66/60 has 10, 8 of which are used strictly for pointers. As a result, implementations of the function `Allocate_registers` may vary with respect to the use of special register symbol nodes and support functions used for allocating registers.

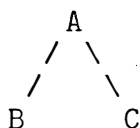
However, for the Nova and the Honeywell 6/30, a register allocation model has been chosen which involves internal constraints imposed on a particular operator as well as external constraints imposed on the subtree of which the operator is the root node. Internal constraints involve the actual register requirements of the machine operations used to implement the operator and also include the specification of where the result of the operation resides. For example, the multiply operation may require that the result of the left subtree (i.e. the left operand of the multiply) must be in the higher numbered register of an arbitrary register pair, that the right operand of the multiply must be in a memory location and that the result of

the multiply is in the higher numbered register of the register pair used with the left operand.

External constraints involve registers already used by the left and right subexpressions and registers used by the left or right sibling of the current operator. In addition there is the specification of a preference set of registers to be imposed on the subexpressions as a result of the internal constraints of an operator. Thus, if the implementation of the addition operator causes the result to appear in the same register as the left operand, then it is meaningful to impose a preference on the left subtree which includes any preference made to the addition operator, since the execution of the addition will leave the result in one of the preferred registers. Similarly, it is meaningful to impose a preference on the right operand which excludes any preference made to the addition operator, since this will aid in the prevention of the right operand from appearing in a register in which the result of the addition is preferred.

Ignoring any preferences made due to internal constraints, preferences made on the left operand usually exclude all registers used by the subexpression of which the right operand is the result, while preferences made on the right operand exclude only the result registers of the left operand. The reason for the above preferences is the left to

right execution of the expression tree. It would be unwise to leave the left operand in a register which may be required to perform some operation in the subexpression which constitutes the right operand. Similarly, one would like to avoid using the result register of the left operand when assigning registers to be used in the subexpression of the right operand. Failure to do so usually results in the use of a temporary to hold the operand result as well as additional instructions to move the result to the temporary location. Experience with register allocation in all of the Code phases implemented so far has shown that a reverse postorder tree walk (right, left, root) can decrease the number of moves which may be required to leave operand results in such temporaries. Consider the following subtree:



Using a postorder walk, the result register for B will be assigned before the result for C. However, the result for B may not remain in that register if the same register is used anywhere in the subexpression at C. This forces a new register to be assigned to the result of subexpression B as well as an extra instruction to move the result to the new register. In order to keep the number of moves to a minimum,

the subexpression at B would need to be re-allocated with registers. By using a reverse postorder walk, the registers used in subexpression C are known and can be avoided when assigning a result register to the subexpression at B.

In general, the task of avoiding registers used by an expression at a right sub-tree occurs only when assigning the result of an expression at the left sub-tree. With a postorder walk, the task of avoiding registers used by the result of a left sub-tree occurs for any and all registers used by the right sub-tree. In some cases avoidance is not possible because of the internal constraints of some of the operators in the right subtree and this results in the movement of results to temporaries.

#### 4.6.3 Machine Instruction Generation

Code generation for expressions is initiated by one of two functions, depending on whether the expression is part of conditional jump. For conditional jumps, the machine-invariant function `Logical_expr_gen` is called. Otherwise the machine-specific function `Arith_expr_gen` is called.

`Logical_expr_gen` is patterned after the procedure first outlined by (Huskey, 1961). Numerous implementations of a

machine-specific nature have been produced; (Wulf, et.al., 1975), (Lowry, 1969). Its basic mode of operation is to perform an In-order walk of the expression tree, changing the target label or sense of the condition as required by the current logical operator. The normally machine-specific operations required for setting condition codes and generating code to retain the truth value of the accompanying expressions are deferred to the machine-specific function `Relation_gen`. When additional labels are required, the machine-invariant function `Int_label` is used to supply a target label for logical subexpressions.

```

Logical_expr_gen( tree, label, jmp_true )

    \ Generate code to evaluate the logical expression tree.
    \ If the value is !0 and jmp_true==TRUE, or if the value
    \ is 0 and jmp_true==FALSE, branch to label
{
    auto t;

    select( OP[ tree ] )
    {
        case IL_NOT:
            Logical_expr_gen( LEFT[tree], label, !jmp_true );

        case IL_LOGAND:
            {
                t = jmp_true ? Int_label() : label;
                Logical_expr_gen( LEFT[tree], t, FALSE );
                Logical_expr_gen( RIGHT[tree], label, jmp_true );
                if( jmp_true ) Backplug_gen( t );
            }

        case IL_LOGOR:
            {
                t = jmp_true ? label : Int_label();
                Logical_expr_gen( LEFT[tree], t, TRUE );
                Logical_expr_gen( RIGHT[tree], label, jmp_true );
                if( !jmp_true ) Backplug_gen( t );
            }

        default:
            Relation_gen( tree, label, jmp_true );
    }
}

```

Relation\_gen is responsible for generating code to evaluate the expression rooted at tree and jump to the label if the result is equivalent to the truth value. Following is a simple implementation of Relation\_gen to illustrate the requirement for a fictitious machine.

```

Relation_gen( tree, label, jmp_true )

    \ If the node is a relational operator, generate code to
    \ perform a comparison. Otherwise the node is a variable
    \ for which condition code can be set explicitly, or an
    \ operator which normally sets condition codes.
{
    auto op;

    select( op=OP[tree] )
    {
        case IL_GT:
        case IL_GTE:
        case IL_LT:
        case IL_LTE:
        case IL_EQU:
        case IL_NEQU:
        {
            Set_condition_register( tree );
            if( jmp_true == FALSE )
                op = Reverse_sense( op );
            Hop_gen( op, label );
        }
        default:
        {
            if( op & OPERATOR_BIT ) Arith_expr_gen( tree );
            else Code( SET, tree ); \ to set indicators
            Hop_gen( jmp_true ? IL_EQU : IL_NEQU, label );
        }
    }
}

Set_condition_register( tree )

    \ Evaluate the left and right subtrees and generate
    \ a compare instruction on the results.
{
    auto left, right;

    Arith_expr_gen( left = LEFT[tree] );
    Arith_expr_gen( right = RIGHT[tree] );
    Code( CMP, RESULTIN[left], RESULTIN[right] );
}

```

The above functions are relatively compact only because of the assumptions made about the machine. For instance, only one possible method of comparing results is alluded to

in `Set_condition_register`. Quite often additional comparison instructions are available, for example, a compare-immediate instruction for comparisons to be made against constants. Additional actions are usually required when the modulus operator is involved. The remainder may result from a division operation, but the condition codes are usually set depending on the quotient, thus requiring additional instructions to explicitly set condition codes for the remainder.

`Arith_expr_gen` is a machine-specific function responsible for walking the expression tree and generating machine code for the operator at each node. A straightforward implementation of `Arith_expr_gen` would involve a large select statement in which each operator is represented by a case. For each case, the appropriate calls to the `Code` function would be made to produce the machine code burst to accomplish the operation. For example, consider the case for the `IL_BITOR` operator as it might be implemented in the TI 990 `Arith_expr_gen`.



```

case IL_BITOR:
{
    if( Is_in_dest( left_result ) )
        Code( .SOC., destination, right_result );
    else if( right_type == CONST_TYPE ||
            right_type == STRING_TYPE )
    {
        if( Reg( destination ) )
        {
            Code( .LI., destination, right_result );
            Code( .SOC., destination, left_result );
        }
        else
        {
            Code( .LI., R[12], right_result );
            Code( .SOC., R[12], left_result );
            Code( .MOV., destination, r[12] );
        }
    }
    else if( Is_in_dest( right_result ) )
        Code( .SOC., destination, left_result );
    else
    {
        Code( .MOV., destination, left_result );
        Code( .SOC., destination, right_result );
    }
}

```

Figure 4.8 Arith\_expr\_gen select case example

Considering the fact that 72 different operators are available, the use of a select statement can result in a very large Arith\_expr\_gen. The example in Figure 4.8 alone results in approximately 120 words of TI 990 code. Some interesting observations can be made about a TI 990 Arith\_expr\_gen implemented as a select statement. In total, approximately 185 tests are made and about 460 actions are performed. There are 16 different types of actions and 19

different types of tests involved. Of the total number of actions, 182 are calls to the Code function and another 93 are calls to one of 34 functions which perform commonly used actions. The total number of tests and actions versus the number of different types of tests and actions has led to the use of tables in an effort to decrease the size of Arith\_expr\_gen. A table can also provide a more concise method of specifying code bursts for operators. The example in Figure 4.8, when expressed as tests and actions, would result in the table entry shown in Figure 4.9.

| <u>Operator</u> | <u>Test</u>          | <u>Action</u>           |
|-----------------|----------------------|-------------------------|
| IL_BITOR;       | left,is_in_dest      | : code .SOC.,dest,right |
|                 | right,const_or_str & |                         |
|                 | dest,in_reg          | : code .LI.,dest,right  |
|                 |                      | code .SOC.,dest,left    |
|                 | right,const_or_str   | : code .LI.,r12,right   |
|                 |                      | code .SOC.,r12,left     |
|                 |                      | code .MOV.,dest,r12     |
|                 | right,is_in_dest     | : code .SOC.,dest,left  |
|                 |                      | : code .MOV.,dest,left  |
|                 |                      | code .SOC.,dest,right   |

Figure 4.9 Code table example

The '&' in the second test is used to indicate that both tests must result in true before the action may be taken. In general, a set of conditions is associated with an

operator. For each condition, composed of zero or more tests, a set of actions is to be performed if all the tests involved in the condition are true (i.e. if the condition is true). If no tests are specified for a condition, the condition is true by default. Only one set of actions is performed, namely, that associated with the first true condition. The set of actions may be empty.

Code tables are maintained in two data structures, `Op_list` and `Conditions_and_actions`. The operator is used as an index into `Op_list` at which an index into `Conditions_and_actions` is found. The content of `Conditions_and_actions[index]` is a pointer to the set of conditions associated with the operator. A condition in the table consists of a sequence of test entries, described in Figure 4.10. If the action count is non-zero, a word containing a pointer to a set of actions immediately follows the sequence.

|       |              |              |           |
|-------|--------------|--------------|-----------|
| ----- |              |              |           |
| X     | action count | test operand | test code |
| ----- |              |              |           |
| 15 14 | 9            | 4            | 0         |

Figure 4.10 Code table test entry

In all but the last test entry, the X field is set to 1 to indicate that another test entry follows. The test code

field contains a number from 1 to 31 which identifies a machine-specific test to be performed. A test code field of zero indicates that the test is true by default. The test operand field contains a number from 0 to 31 which identifies an argument to be used in the test. The action count indicates the number of actions in the set associated with the condition. A count is used rather than an end-of-action indicator so that the indication would not be required at the end of each set of actions. In addition to saving one word at the end of each set of actions, using a count prevents an action code from having to be set aside for the purpose of indicating the end of the actions.

A set of actions in the table is represented by a sequence of one or two word action entries, as described in Figure 4.11. A non-zero X field in an action entry indicates a two word action entry in which the second word contains a full target word operand.

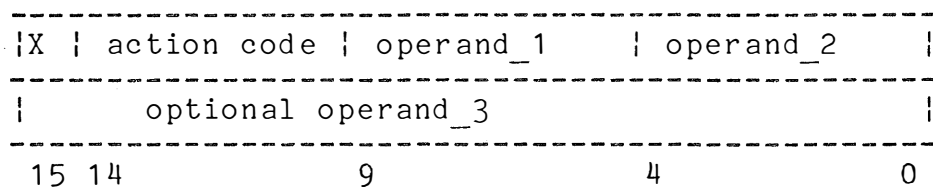


Figure 4.11 Code table action entry

The two fields operand\_1 and operand\_2 contain a value from

0 to 31 which identify the same set of arguments as those referenced by a test entry. The optional operand\_3 may contain any 1-word value. The action code field contains a value from 0 to 31 which identifies an action to be performed using the arguments specified in the action entry. An action code of zero indicates that the Code function is to be called where operand\_3 is the machine instruction to be generated. An action code of one indicates that conditions are to be tested and actions performed for the operator code contained in operand\_3. This is analagous to a subroutine call and is used in situations where the same kind of tests and actions must be performed for a large number of operators. An example would be the generation of code to move the result of a machine operation to the correct destination if it is not already there. The manifests CODE and CALL have the values 0 and 1 respectively and are supplied with the machine-invariant part of Code for use in code tables. To facilitate the passing of operands to a CALL action, a third data structure, called Action\_stack, is maintained. The operand\_1 field of a CALL action contains a value by which a pointer, Action\_stack\_ptr, is incremented by the machine-invariant function Test\_and\_act. Test\_and\_act, called by Arith\_expr\_gen to process an operator, as well as the machine-invariant function

Check\_conditions used by Test\_and\_act, are discussed later in this section. Given the sizes of the various code table entries, the total number of words required to implement the code table for Figure 4.9 is 29 as compared to 120 words of TI990 code, a substantial saving in space over the method of using a select statement case. Note also that the number of words used in the table remains constant regardless of the host machine, unlike the use of host machine code to execute an equivalent select case statement.

Three machine-specific functions are required to aid in the generation of code using code tables; Expand, Test, and Action.

Expand is used to provide the address of a symbol (or pseudo symbol) table node which corresponds to an operand in a test or action entry.

Test is used determine the truth of a test entry. It returns true or false depending on whether the test operand satisfies the test specified by the test code.

Action is used to perform one of the remaining 30 actions specified by the action code in an action entry. The procedure of testing conditions and initiating actions is performed with the aid of two machine-invariant functions, Test\_and\_act and Check\_conditions. The function Test\_and\_act uses Check\_conditions to find the first true condition and

its associated set of actions.

Check\_conditions is used to find the first true condition. It uses the Test function to determine the truth of each test in a condition. On the first condition in which all tests are true, Check\_conditions sets the information regarding the number of actions in the action set associated with the true condition. A list of the Condition, Action, and Expand functions as they apply to the TI 990 implementation can be found in Appendix B.

```

Test_and_act( operator, nargs )

    \ Increment Action_stack_ptr and Argument_ptr by the
    \ value in nargs.
    \ Call Check_conditions to perform the tests and use
    \ information set up by Check_conditions to perform
    \ the required action.
    \ If the action is CALL, call Test_and_act.
    \ If the action is CODE, call Code.
    \ Otherwise, call Action to perform the actions.
{
    extrn Action_stack_ptr, Argument_ptr, Action_info;
    auto op1, op2, action_ptr, action_count;
    auto action_word, old_args;

    old_args = Argument_ptr;
    Argument_ptr = Action_stack_ptr;
    Action_stack_ptr += nargs;

    Check_conditions( operator );
    action_ptr = ACTION_PTR[Action_info];
    action_count = ACTION_COUNT[Action_info];

    while( action_count-- )
    {
        action_word = *action_ptr++;
        operator = action_word & OP_CODE >> 10;
        op1 = Expand( action_word & OP1 >> 5 );
        op2 = Expand( action_word & OP2 );

        select(operator)
        {
            case CALL:
                Test_and_act( *action_ptr++,
                               action_word & OP1 >> 5 );
            case CODE:
                Code( *action_ptr++, *op1, *op2 );
            default:
                Action( operator, op1, op2,
                        action_word & OP3_EXISTS ?
                        *action_ptr++ : 0 );
        }
    }

    Action_stack_ptr = Argument_ptr;
    Argument_ptr = old_args;
}

```



```
Check_conditions( op )
```

```

\ Until the first true condition is found, call Test to
\ perform the tests for a condition.
\ If any test is false, skip the remaining tests and
\ action pointer (if any) and check the next condition.
\ Set up the action count and the pointer to the set
\ of actions when the true condition is found.
{
extrn Conditions_and_actions, Action_info, Op_list;
auto  condition,result, condition_ptr;

condition_ptr=Conditions_and_actions[ Op_list[op>>1] ];

repeat
{
    condition = *condition_ptr++;
    ACTION_COUNT[Action_info] = condition&COUNT >> 10;

    if( op = condition&TEST_CODE )
    {
        result = Test( op,
                        *Expand(condition&TEST_OP>>5) );

        while( result==TRUE && condition&CONTINUE )
        {
            condition = *condition_ptr++;
            result&=Test( condition&TEST_CODE,
                          *Expand(condition&TEST_OP>>5));
        }
    }
    else result = TRUE;

    if( result==TRUE )
    {
        ACTION_PTR[Action] = *condition_ptr;
        return;
    }
    while( condition&CONTINUE ) \ skip remaining tests
        condition = *condition_ptr++;

    if( ACTION_COUNT[Action_info] ) ++condition_ptr;
    \ skip action pointer
}
}

```

There are some operators which cannot be implemented using code tables, or which are better implemented as a select case within Arith\_expr\_gen. One of these is the query operator, IL\_QUERY, for which a sample coding is shown below. The problem here is one of passing labels from the '?' part of the operator to the ':' part of the operator. The relational and logical operators also cause problems because of the need to call on Logical\_expr\_gen to handle the subexpression while at the same time generating code which will retain the truth value of the expression. The following example code also shows how relational and logical operators might be handled within Arith\_expr\_gen.

```

case IL_QUERY:
{
    false = Int_label();
    destination = RESULTIN[tree];
    Logical_expr_gen( LEFT[tree], false, FALSE );

    result = RESULTIN[ subtree=LEFT[RIGHT[[tree]] ]];
    Arith_expr_gen( subtree );
    if( result != destination )
        Code( MOVE, result, destination );
    Jump_gen( done = Int_label() );

    Backplug_gen( false );
    result = RESULTIN[ subtree=RIGHT[RIGHT[tree]] ];
    Arith_expr_gen( subtree );
    if( result != destination )
        Code( MOVE, result, destination );
    Backplug_gen( done );
}

```

```

case IL_GT:
case IL_GTE:
case IL_LT:
case IL_LTE:
case IL_EQU:
case IL_NEQU:
case IL_NOT:
case IL_LOGOR:
case IL_LOGAND:
{
    destination = RESULTIN[tree];
    label = Int_label();
    Code( CLR, destination ); \ default to false
    Logical_expr_gen( tree, label, FALSE );
    Code( INC, destination ); \ set to TRUE
    Backplug_gen( label );
}

```

The use of a table of machine instructions indexed by conditions is not new; a table of "coding skeletons" was used in the IBM Fortran H compiler (Lowry, 1969). The table was indexed by the arithmetic operator, and the resulting subtable indexed by conditions concerning the operands (in register, memory, etc.). The implementation of code tables in Code, however, is considerably less machine-specific in its structure.

The use of code tables results in a considerable saving in code space. For the TI 990, Arith\_expr\_gen implemented solely with select cases is approximately 6000 words larger on a TI 990 host than Arith\_expr\_gen implemented using code tables. The code tables use approximately 1200 words, thus resulting in an overall saving of about 4800 words. The total of the TI 990 Code phase on a TI 990 host is currently

around 20,500 words. An increase in execution time can be expected using code tables because of the need to call functions to perform tests and actions which would normally be handled in `Arith_expr_gen` select cases. However, only about one percent of the total execution time for the TI 990 Code phase, is used to execute the Test and Action functions. A significantly larger percentage (5 percent) of the total execution time is spent on `Test_and_act` and `Check_conditions`. Unfortunately, no figures are available for the amount of time spent by an `Arith_expr_gen` implemented using only select cases. As a result it is difficult to determine what the total increase in execution time may be.

#### 4.7 Control-path Support

Control-path support is the collection of functions used to aid in the generation of machine instructions which reference labels. Three machine-invariant functions are available for control-path support.

`Defer_gen` is a function which saves information concerning label referencing instructions which cannot be generated because the label has not yet been defined (i.e. the generation is deferred until a later time). `Defer_gen` leaves a gap in the code segment, which is large enough so

that once the label is defined, the instruction may be generated in the gap.

Backplug\_gen is a function which is called when a label is defined. It generates all instructions which have been deferred by inserting the instructions in the proper gaps.

Check\_addressability is a function which is used to determine if deferred instructions can no longer reference their respective labels due to some addressing constraint. It uses a machine-specific function, Backplug\_symbol, to modify or replace the deferred instruction.

Label referencing instructions are usually those which cause branches or conditional jumps in the execution of a function module. They may also be instructions which manipulate data such as literal constants or pointers which must be stored in memory, thereby referencing labels which are associated with the location of data. A straightforward way of handling data referencing instructions is to generate the data using the approach used in the Data phase. That is, generate the data in a data segment and use its location and the Wsd index of the associated local symbol to relocate the address field of the data referencing instruction. Unfortunately, this approach is not always possible in machines which are constrained in their ability to address memory. In particular, this is true of the NOVA, in which

all address specification is limited to an 8 bit displacement and 2 bit modifier. The result is an absolute address in the range 0 to 255, or, an index range of -128 to +127 relative to the program counter or register 2 or 3. Because the number of data words required by a function module may be more than 256, the use of absolute addresses is not recommended. The use of indices themselves is not desirable in a machine with few index registers because of the potential inefficiency of the generated code caused by saving and restoring the addresses pointing to the data areas. This leaves addressing relative to the program counter (PC) as the only reasonable alternative.

The problems associated with PC-relative addressing become particularly acute when forward referenced labels are involved. It is conceivable to think of a two pass system of generating code in which the first pass is used to tag label referencing instructions so that the addressability to their respective labels may be ascertained and corrected in the second pass. At the very least, this necessitates maintaining a linked list structure of all label referencing instructions, as exists in the Bliss compiler (Wulf, et. al., 1975), because the instructions cannot be generated until the second pass.

A considerable amount of literature has been published

concerning the problems of generating machine code for machines with instructions which depend on the distance from the operand for their final instruction size. A recent paper by Szymanski (1978) discusses an optimal, two pass algorithm which minimizes the total size of code containing such instructions. A major point concerns the decision to use the short form of such instructions as a first step toward decreasing total code size. The decision to use short forms was also made for the Eh compiler, however, the desire to stay away from two pass algorithms to solve the size problem has resulted in the current method of leaving a gap in the load code where a forward referencing instruction would be generated. The gap is filled only when the target label of the instruction becomes defined or when the instruction itself may soon lose addressability to the target label. Since the target may in some cases be too far away from the instruction and since only enough room is left in the gap for the original instruction, the cost of correcting the situation may seem high. The correction method involves specifying an machine-specific distance at which code generation may not continue until some corrective measures have been taken with respect to the instruction which may soon be unable to address its label. Such corrective measures may include the generation of a branch instruction

to the label in question (ie. a long format instruction which can always reach its target label) to which the original instruction has been modified to jump. Where indirection is allowed, the corrective action may be to reserve a word at the current code segment location at which point the address of the label will be placed once it is defined. The original instruction is then modified to indirect through this location in order to reach the target label.

In either case, the corrective actions usually interrupt the generation of code which was current at the time the correction was deemed necessary. This results in additional instructions being generated so that the code execution will jump around the correction. The ironic part is that the label may be defined immediately after the correction has been made, thus wasting code space and execution time in the final core-image.

With these points in mind, it is interesting to consider the number of corrections which result using the TI 990 compiler while compiling the Thoth operating system. Out of a total number of 1121 forward referenced jumps using the short format, only 25 needed to be corrected by modifying them to jump to a long format branch instruction. Given that two words of overhead result from each correction (one for



the jump around the correction and one for the original instruction which might have been replaced by the long form in the first place) a total of 50 words are involved out of the 11,694 words of code generated for the system.

A point of interest concerns the correction procedure itself. If any instruction needs correction, all instructions which reference the same label (up to the point at which the correction is being made) are corrected. Consider, for example, four such instructions which cannot reach their common target label. If it were known that they cannot reach the label, then the long form would have been used, for a total of 8 words of code. However since it is not known that the instructions cannot reach the label, and since the short form of the instruction would be used in such cases, only 7 words are required: 4 for the original instructions, 2 for the correction (a branch) and 1 for the jump around the correction. In terms of space, then, it seems more efficient to use this correction method once three or more instructions are deferred to the same label.

The information used to make such corrections is maintained in a data structure called the defer queue.

#### 4.7.1 Defer Queue

The defer queue is a multi-linked list of instructions which have been deferred for lack of a defined label. A full-sized defer queue node is outlined in Figure 4.12. The size of a defer queue node is ultimately defined machine-specifically by means of manifests which conditionally include certain fields.

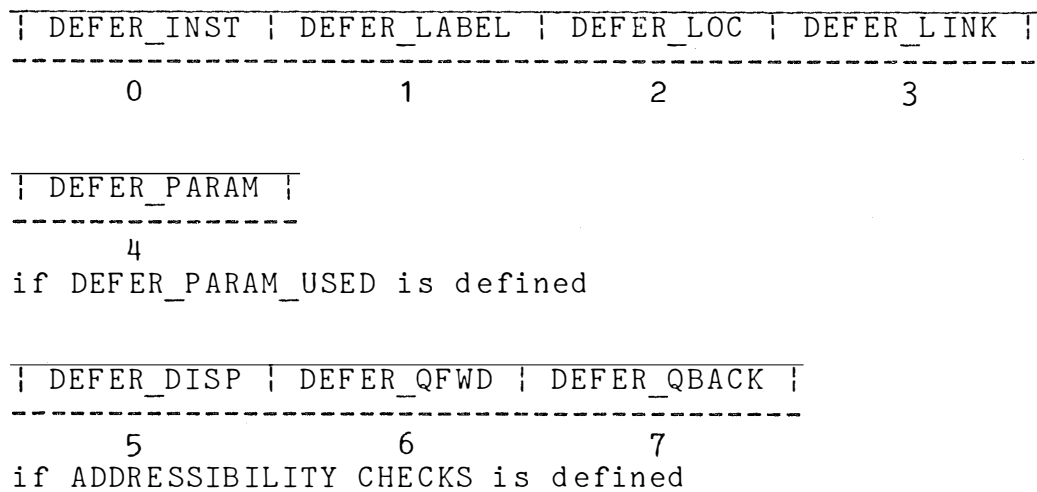


Figure 4.12 Defer queue entry

If the target machine is one which has addressing constraints which must be observed, the manifest ADDRESSIBILITY\_CHECKS must be defined so that the DEFER\_DISP, DEFER\_QFWD and DEFER\_QBACK fields are included in a defer queue node. DEFER\_QFWD and DEFER\_QBACK are used to link all defer queue nodes together. DEFER\_DISP is used

to hold the highest location counter value which can be tolerated before some alternate means of referencing the label must be attempted. DEFER\_PARAM is defined only if the manifest DEFER\_PARAM\_USED is defined. This field is used primarily for those machines which need an optional parameter passed to the Code function from deferred instructions. While most deferred instructions tend to be jumps which reference labels, others, such as register loads or jumps conditional on register contents, require a pointer to a machine-specific register type symbol to be included in the defer queue node.

The remainder of the node describes particulars about the deferred instructions which must always be known in order for them to be generated when the label becomes defined. DEFER\_INST contains the machine instruction which was deferred at the code segment address saved in DEFER\_LOC, and which references the label pointed to by DEFER\_LABEL. DEFER\_LINK is used to link all defer queue nodes which reference a particular label. The VALUE field of the symbol table entry for the label is used to point to the head of the list of its defer queue entries.

#### 4.7.2 Deferring Instructions

The operation of deferring instructions is performed by `Defer_gen`. `Defer_gen` is a machine-invariant function which adds a defer node to the list of defer nodes headed by a referenced label. If addressing constraints are involved, the node is also added to the doubly linked list of all defer nodes. The method of entry into the doubly linked list is such that the list is sorted in ascending order by the values in the `DEFER_DISP` fields. That is, the order of the defer nodes reflects the order in which the the label referencing instructions are to be corrected if addressability to the target label is in danger of being lost.

In addition to the arguments which specify the instruction to be deferred and the symbol node to which it is being deferred, a displacement argument is used to calculate the highest location counter tolerable before steps must be taken to ensure that the instruction will still be able to address its label. The value of the displacement is calculated using a number of variables:

- 1 `INST_DISP` - the addressing constraint (in bytes) of the instruction being deferred. That is, the largest forward reference allowed by the instruction.
- 2 `CORRECTION_DISP` - the size (in bytes) of the code burst

which is generated to correct one occurrence of the possible loss of the addressability.

- 3 JUMP\_DISP - the size (in bytes) of the jump instruction which may be generated to bypass the corrections.
- 4 CODE\_BURST\_DISP - the maximum size (in bytes) of a code burst which may not have correction code inserted. That is, the maximum size of a code burst which can be generated before Check\_addressability can be called to do its job. In most cases, this is the maximum size of a machine instruction, however certain code bursts, such as interrupt enables on the TI 990, use more than one instruction as well as a known relative address. In such cases, the insertion of correction code will result in incorrect execution of the final core-image.

The displacement parameter is then simply calculated as INST\_DISP less the sum of CORRECTION\_DISP, JUMP\_DISP and CODE\_BURST\_DISP. It would be convenient if the displacement value were such that all instructions with addressing constraints could be guaranteed to reach their respective labels, whether corrections were used or not. In fact, such assurances can be made provided that the size of the correction code does not exceed the size of the instruction being corrected. This can be explained by considering the worst case example of deferring two adjacent instructions

which cannot reach their respective, unique, labels without incurring correction code. In such a situation, the second instruction has a highest tolerable address which is greater than the first instruction's highest tolerable address by the size of the first instruction. If the first instruction is then corrected by using code which is greater than the first instruction, the second instruction will have lost addressability to its label because its highest tolerable address is occupied by correction code for the first instruction.

```

Defer_gen( instruction, symbol, length, displacement; param )

    \ Defer generation of the instruction
    \ until the symbol becomes defined.
{
    extrn Counter, Defer_queue_tail, Defer_queue_head;
    auto i, limit, loc, p;

    p = .Alloc_vec( DEFER_NODE_SIZE );

    DEFER_INST[p] = instruction;
    DEFER_SYMBOL[p] = symbol;
    DEFER_LOC[p] = *Counter;      \ remember where to
                                \ backplug it
    *Counter += length; \ leave a gap for backplugging
    Flush_load_bufs();

    DEFER_LINK[p] = VALUE[symbol]; \ link it into the list
    VALUE[symbol] = p;             \ of deferred instructions
                                    \ for this symbol

    \ calculate maximum tolerable location counter

    loc = DEFER_DISP[p] = displacement+DEFER_LOC[p];
    limit = &Defer_queue_tail - DEFER_QFWD;

    \ sort by displaced locations

    for( i=Defer_queue_head; i!=limit; i=DEFER_QBACK[i] )
        if( DEFER_DISP[i] >= loc ) break;

    DEFER_QBACK[p] = i;           \ link it in
    DEFER_QFWD[p] = DEFER_QFWD[i];
    DEFER_QBACK[DEFER_QFWD[i]] = p;
    DEFER_QFWD[i] = p;

    if( .Nargs() == 5 )
        DEFER_PARAM[p] = param;

    return( p );
}

```

#### 4.7.3 Backplugging Instructions

Once a label is defined, any instructions which have been deferred to the label are generated using `Backplug_gen`. `Backplug_gen` is a machine-invariant function designed to generate all deferred instructions associated with the symbol given as the argument. Symbols are not necessarily limited to labels although `Funct_gen` calls `Backplug_gen` for each label definition only. For instance, in the NOVA Code phase, `Backplug_gen` is also used for defining the location of constants and addresses.

`Backplug_gen` sets the `SYMBOL_DEFINED` bit in the symbol node, sets the `VALUE` field to the value of the location counter for the code segment and calls `Code` to generate the instructions found in the accompanying defer list. The code segment location counter is set to the value in `DEFER_LOC` before `Code` is called. The external variable `Backplugging` is set so that it may be used to prevent other routines (such as those which may be called by `Code`) from calling `Backplug_gen`. The variable `Backplugging` can also be used by other machine-specific functions to prevent `Check_addressability` from ultimately inserting correction code.



```

Backplug_gen( symbol )

    \ Backplug instructions which depend
    \ on the symbol just defined
{
    extrn Counter, Backplugging;
    auto  p, dlist, save;

    dlist = VALUE[symbol]; \ list of deferred instructions
    VALUE[symbol] = *Counter/MACHADDR_FACTOR;
    TYPE[symbol] |= SYMBOL_DEFINED;
    save = Counter;
    Backplugging = TRUE;

    while( p=dlist )
    {
        if( DEFER_QFWD[p] ) \ still in queue, so remove it
        {
            DEFER_QBACK[DEFER_QFWD[p]] = DEFER_QBACK[p];
            DEFER_QFWD[DEFER_QBACK[p]] = DEFER_QFWD[p];
        }
        Counter = &DEFER_LOC[p];
        Flush_load_bufs();

        if( DEFER_PARAM[p] )
            Code( DEFER_INST[p], DEFER_PARAM[p], symbol );
        else
            Code( DEFER_INST[p], symbol );
        dlist = DEFER_LINK[p];
        .Free( p );
    }
    Backplugging = FALSE;
    Counter = save;
    Flush_load_bufs();
}

```

#### 4.7.4 Checking Symbol Addressability

Although the need to check addressability of symbols is machine-specific, the method of checking is machine-invariant using `Check_addressability`. The method involves the following steps:

1. If the variable Backplugging is set to TRUE, do nothing.
2. Compare the code segment location counter with the DEFER\_DISP field of the defer node at the head of the defer queue. If DEFER\_DISP is less than the counter, no more checking is required since all other DEFER\_DISP fields will be less than the counter.
3. Call the machine-specific function Backplug\_symbol on DEFER\_SYMBOL to modify or replace the deferred instruction. Call Backplug\_symbol on any additional deferred instructions for which the DEFER\_SYMBOL is not a label and the DEFER\_DISP is within a specified range from the location counter or the DEFER\_SYMBOL is a label and the DEFER\_DISP is greater than or equal to the location counter.

The variable Backplugging is checked to ensure that Check\_addressability is not being called as a result of actions initiated by Backplug\_gen. Since the location counter would be altered in such situations, it is meaningless to check symbol addressability. Check\_addressability is called with an optional parameter which is used to determine if a jump should be generated around any code which Backplug\_symbol may generate.

```

Check_addressability( ; skip, range )

    \ If the first entry in the defer queue needs no
    \ backplugging exit. Otherwise, backplug the
    \ instruction and any others whose target symbol
    \ is a non label and which falls within the
    \ specified range. If skip == 1 or is not given
    \ generate a jump around any code which might be
    \ generated.
{
    extrn Backplugging, Defer_queue_head;
    extrn Defer_queue_empty, Counter;
    auto deferred, previous, symbol;

    deferred = Defer_queue_head;
    if( deferred != Defer_queue_empty &&
        *Counter >= DEFER_DISP[deferred] &&
        !Backplugging )
    {
        if( .Nargs() < 1 ) skip = TRUE;
        if( .Nargs() < 2 ) range = 0;
        if( skip ) Jump_gen( skip=Int_label() );

        Backplug_symbol( DEFER_SYMBOL[deferred] );
        previous = &Defer_queue_head - DEFER_QFWD;
        while( (deferred=DEFER_QFWD[previous]) !=
            Defer_queue_empty )
        {
            if( *Counter < DEFER_DISP[deferred]-range ) break;
            symbol = DEFER_SYMBOL[deferred];
            if( TYPE[symbol]&TYPE_FIELD == LABEL_TYPE &&
                *Counter < DEFER_DISP[deferred] )
            {
                previous = deferred;
                next;
            }
            Backplug_symbol( symbol );
        }
        if( skip ) Backplug_gen( skip );
    }
}

```

The operations performed to correct the addressability of a symbol vary with the target machine thus requiring Backplug\_symbol to be machine-specific. For instance, on the

TI, the only instructions which are constrained are conditional jumps. The target address of the jump cannot be addressed indirectly so the constraint is circumvented by generating a branch instruction to the original label. All jumps which have been deferred to that label are then backplugged to jump to the branch instruction.

```

Backplug_symbol( symbol ) \ for the TI 990
{
    auto label;

    label = Int label();
    VALUE[label] = VALUE[symbol]; \ copy defer queue pointer
    VALUE[symbol] = 0;
    Backplug_gen( label );
    Branch_gen( symbol );
}
Branch_gen( label ) \ for the TI 990

    \ Emit a branch instruction if label defined.
    \ Otherwise defer it with address in range.
{
    auto p;

    if( !(TYPE[label]&SYMBOL_DEFINED) )
    {
        p = Defer_gen( .B., label, 4, 240 );

        \ Now disconnect it so Check_addressability
        \ will not find it in the queue

        DEFER_QBACK[DEFER_QFWD[p]] = DEFER_QBACK[p];
        DEFER_QFWD[DEFER_QBACK[p]] = DEFER_QFWD[p];
        DEFER_QFWD[p] = 0;
    }
    else Code( .B., label );
}

```

The Backplug\_symbol function is very simple on the TI 990 because there are few addressing constraints on the TI

990. The Nova, however, is heavily constrained in its addressing capability. Not only are all jumps limited to a PC-relative range, but due to the method of addressing constants within a function module, such data references are similarly constrained.

Address\_gen( symbol, indirect ) \ for the Nova

```

\ Generate a pseudo instruction (.ADDR.) which specifies
\ to Code that only the address of the symbol is to be
\ generated. If the symbol is not yet defined, defer the
\ instruction and remove it from consideration by
\ Check_addressability.
\ The argument indirect modifies .ADDR. to signify whether
\ the indirect bit is to be set by Code.
{
  auto p, opcode;

  opcode = .ADDR. | indirect;
  if( !(TYPE[symbol]&SYMBOL_DEFINED) )
  {
    p = Defer_gen( opcode, symbol, 2, 230 );

    \ Prevent Check_addressability from finding it.

    DEFER_QBACK[DEFER_QFWD[p]] = DEFER_QBACK[p];
    DEFER_QFWD[DEFER_QBACK[p]] = DEFER_QFWD[p];
    DEFER_QFWD[p] = 0;
  }
  else Code( opcode, symbol );
}

```

Backplug\_symbol( symbol ) \ for the Nova

```

\ If the symbol is a label, modify all its deferred
\ instructions to reference the symbol indirectly
\ and defer the address of the label.
\ Otherwise generate the value of the symbol.
{
  auto type, label, rel_wsd, value, ind_label;
  auto prev, ptr;

  ind_label = label = 0;
  if( (type=TYPE[symbol]&TYPE_FIELD) == LABEL_TYPE )
  {
    prev = VALUE + symbol - DEFER_LINK;
    while( ptr = DEFER_LINK[prev] )
    {
      if( DEFER_INST[ptr]&$F800 == .ADDR. )
      {
        prev = ptr;
        next;
      }
      if( DEFER_INST[ptr] & .INDIR. )
      {
        \ Already uses indirection, so set it aside.

        if( !ind_label ) ind_label = Tree_label();
        DEFER_SYMBOL[ptr] = ind_label;
        DEFER_LINK[prev] = DEFER_LINK[ptr];
        DEFER_LINK[ptr] = VALUE[ind_label];
        VALUE[ind_label] = ptr;
      }
    }
    else
    {
      DEFER_INST[ptr] |= .INDIR.;
      if( !label ) label = Tree_label();
      DEFER_SYMBOL[ptr] = label;
      DEFER_LINK[prev] = DEFER_LINK[ptr];
      DEFER_LINK[ptr] = VALUE[label];
      VALUE[label] = ptr;
    }
  }
  if( label )
  {
    Backplug_gen( label );
    Address_gen( symbol, FALSE );
  }
  if( ind_label )
  {

```

```

        Backplug_gen( ind_label );
        Address_gen( symbol, TRUE );
    }
    return;
}

\ Handle non-label references here.

Backplug_gen( symbol );
select( type )
{
    case CONST_TYPE:
    case ADDRESS_TYPE: value = CONST_VALUE[symbol];

    default: value = 0;
}

rel_wsd = TYPE[symbol]&REL_WSD_FIELD;

if( !rel_wsd ) Load_word( value );
else Rload_word( rel_wsd>>WSD_BITS,
                rel_wsd&WSD_FIELD,
                value );
}

```

Note that the Nova Backplug\_symbol makes use of .ADDR., a pseudo machine opcode. This is used so the Code function for the Nova can recognize the fact that only the address needs to be generated when Backplug\_gen is finally called to define the original label.

The Code function for the Nova must also recognize when a symbol which has been previously defined loses its addressability. This happens, for instance, when a label exists but the instruction being generated is more than 127 words past the location of the label. It can also happen when a constant symbol has been generated but an instruction which loads the constant into a register is more than 127

words past the location of the constant. Although these are both instances of the same problem, the solution varies with the symbol type. Labels can only be associated with one location in the code module, whereas constants may be associated with a number of locations, each containing the value of the constant symbol. Thus, when a constant loses its addressability, Code would simply defer the instruction, and indicate that the constant symbol is once again undefined. Eventually, the symbol would be processed by `Check_addressability` and defined once again by `Backplug_symbol` which would also generate the value of the constant.

Unfortunately, the same cannot be done for a label symbol because it exists in only one location in the code module. However, the location of the label symbol is known and can be used to set the `ADDRESS_VALUE` field of a special address symbol through which the label may be referenced indirectly. The address symbol is somewhat similar to a constant symbol in that it has an `ADDRESS_VALUE` field which is a constant offset from the beginning of the code module. The constant offset is later relocated by `Load` when the module is loaded. Therefore, the problem of losing addressability to a label is solved by the replacement of the label operand of an instruction with a special constant



operand. The Nova Code function makes a replacement in such situations, and defers the instruction, which is eventually processed by Check\_addressability.

To be effective, Check\_addressability should be called as often as possible, preferably before each instruction is generated. It is therefore the first function called within the Code function.

#### 4.8 Portability

The portability of Code is enhanced by the heavy use of manifests. As with Data, the use of manifests in Code has resulted in a large part of the source being effectively machine-invariant. The structure of Code is such that machine-specific requirements are highly visible and localized, thus further reducing the effort of implementing and porting the compiler. The compiler has been ported from the Honeywell 66/60 to the TI 990. On both machines it produces code for the TI 990, the Data General Nova and the Honeywell 6/30. On the Honeywell 66/60 it also produces code for the Honeywell 66/60.

To date, the only major problem in porting the compiler occurred in the use of 16-bit negative quantities on a machine with 36-bit words. Since the quantities appeared positive in the 36-bit words, binary search tables for cross

compiled programs were being built incorrectly. No source changes are required in order for Code to execute on a different host, hence Code is machine-independent.

## Chapter 5

### Conclusions

This thesis describes the structure of the code generation phases of the Eh compiler. A focal point of the thesis is the machine-invariant aspect of Code and Data. The lack of machine-invariance in some aspect of code generation will usually result in a machine-specific solution, however in many cases machine-invariance can be introduced by careful distinction between the machine-invariant and machine-specific properties of the particular aspect. It is the contention of this thesis that the structure of Code and Data has been highly successful in maintaining such distinctions.

The interfaces between machine-invariant and machine-specific operations are well defined and have been kept to a minimum, the most important being calls to machine-specific functions such as Modify, Allocate\_registers and Arith\_expr\_gen. An important point to consider is that the concept of machine-invariance does not disappear once a machine-specific function is called. For example, calling the machine-specific function Arith\_expr\_gen makes use of machine-invariant functions for

manipulating code tables. These functions, in turn, make use of machine-specific functions such as `Test` or `Code` which ultimately use machine-invariant functions to produce relocatable load code. Similar interactions exist in control-path support and select statement support.

The structure of `Funct_gen`, while being machine-invariant, is also flexible with respect to the intermediate language input. An example is the recent addition of keyword processing (see Section 4.1.4). The decision to split the primary code generation operations into the above three functions has encouraged research into the problems of register allocation and code generation by isolating the problem areas. The result has been good quality code for at least four architecturally diverse machines.

The use of relocated stack frame parts has fostered good stack designs which would not have been possible without introducing a second pass to determine the size of stack space for temporaries. Perhaps a better conclusion to be drawn from this is that the design of the load code (see Chapter 2) is highly suited to the needs of code generation, in this case, the use of relocatability.

The above comment on the load code also applies to control-path support. Without the ability to "backplug"

instructions, the algorithms would become even more complex or two passes might be required. In addition to deferring code generation, control-path support provides an alternative method of handling addressing constraints along with a guide for ensuring that instructions with such constraints can be generated correctly. The ability to handle addressing constraints for one target machine does not impose unnecessary overhead when generating code for target machines which have no such constraints because the ability has been made optional.

The introduction of a table driven mechanism for generating machine instructions, has resulted in a means by which alternate code bursts for expression operators may be implemented in a relatively short period of time. It is easier to understand how code is generated for a particular machine when the pertinent information is represented tabularly. Additionally, code tables for new machines become easy to implement when an existing table can be used as a guide.

The structure of Code and Data has led to a reasonably high level of portability. Porting the compiler has taken from 2 to 4 months depending on the complexity of the target machine and the expertise of the person doing the port. The Nova compiler was recently re-implemented to bring it up to

date the TI 990 and Honeywell 6/30 Code phases. Approximately 5 weeks were required by the author to re-implement the Nova Code phase from scratch. Another 3 weeks were spent in debugging. These figures would have been lower if there had not been simultaneous additional developments in the register allocation algorithms.

Finally, the Data phase shows the extent to which code generation for data structures can be machine-invariant. The machine-specific nature of such code generation has been reduced to about 10 variables currently specified as manifests which are also used by Code. An immediate benefit of a separate phase is the ability to generate load code for data modules concurrently with the execution of Clean and Code.

### 5.1 Future Research

Because the nature of code generation is ultimately machine-specific, certain properties assumed by machine-invariant operations may not be true for some new machine (new, in the sense that the machine has come under consideration for a port rather than necessarily being a recent machine design).

One such assumption concerns the notion that addressing constraints can be expressed in terms of some contiguous

addressable space (i.e. there are no holes). The Honeywell 6/30 already invalidates the assumption by disallowing jumps to the next instruction (effectively a no-op instruction). The Hewlett Packard 21MXE presents a greater problem in that the addressing constraints for relative addresses include the requirement to remain within the logical memory page in which the instruction is found. Currently, control-path support is not equipped to handle such situations, indicating the need for more research into the problems of addressing constraints and possible machine-invariant solutions.

The problem of register allocation is one research area which is currently being worked on. Once again, the emphasis is on the design of machine-invariant algorithms and data structures which can be used with the machine-specific properties of register usage.

Another area for future research concerns the possibility of grouping statements into blocks of code for which register usage can be highly optimized. Currently, code is generated on a statement by statement level due, in part, to the lack of a rudimentary flow analysis for determining the effect on register usage when side effects are present in the module being compiled. For example, if the use of a variable involves a register load, the

knowledge of a register already containing the value of the variable can save a register load provided that the value of the variable has not changed since it was loaded into the register. Extra register loads involved in the use of the keyword **active** account for about 5 percent of the code space in the TI 990 implementation of the Thoth operating system.



## Bibliography

- Braga, R.S.C.(1976), "Eh Reference Manual", Research report, CS-76-45, University of Waterloo, Dept. of Computer Science, November.
- Cheriton, D.R., M.A. Malcolm, L.S. Melen, and G.R. Sager(1978), "Thoth, a Portable Real-time Operating System", Research report, CS-78-15, University of Waterloo, Dept. of Computer Science, March.
- Sager, G.R.(1977), "The Thoth Linking Loader", Research report, CS-77-15, University of Waterloo, Dept. of Computer Science, January.
- Johnson, S.C. and B.W. Kernighan(1973), "The Programming Language B", Bell Laboratories Computing Science Technical Report, No. 8, January.
- Richards, M.(1969), "BCPL: a Tool for Compiler Writing and System Programming", Proc. Spring Joint Computer Conference.
- Lowry, E.S. and C.W. Medlock(1969), "Object Code Optimization", CACM 12.1, January.
- Huskey, H.D. and W.H. Wattenburg(1961), "Compiling Techniques for Boolean Expressions and Conditional Statements in Algol 60", CACM 4.1, January.
- Wulf, W., R.K. Johnson, C.B. Weinstock, S.O. Hobbs and C.M. Geschke(1975), "The Design of an Optimizing Compiler", American Elsevier, New York.
- Stafford, G.J.(1978), "The Structure of the Eh compiler: Lexical Analysis, Parsing and Optimization", Masters Thesis, University of Waterloo, Dept. of Computer Science, April.
- Szymanski, T.G.(1978), "Assembling Code for Machines with Span-Dependant Instructions", Communications of the ACM 21, 4 (April 1978).
- Knuth, D.E.(1973), "The Art of Computer Programming, Vol. 3, Sorting and Searching", Addison-Wesley, Reading, Mass.

## Appendix A

### Code and Data Input

The following grammars have been copied, with permission, from Stafford (1978).

#### A.1 Externals\_parsil

The information found in the Externals\_parsil simply convey the external variable declarations found in the source program to Data.

```
file           := module_list '*0'
                | '*0'

module_list    := module module_list
                | module

module         := simple_declaration
                | vector_declaration
                | string_declaration

simple_declaration := ':' name_id RELOC data_word

vector_declaration := '[' name_id RELOC initial_list '*0'

string_declaration := '"' name_id RELOC length string_id

data_word      := 'g' name_id
                | CONSTANT_VALUE

initial_list   := initializer initial_list
                |

initializer    := data_word
                | '"' string_id
```

```

| '[' initializer initial_list ']'
name_id      := ID
string_id    := ID
length       := CONSTANT_VALUE

```

To a person familiar with the grammar of **Eh** the correspondence with the source should be obvious in most cases. It should be pointed out however, that the construct 'g' name\_id, is used to represent a reference to an external variable.

## A.2 Cleanil

The following is a description of the input to **Code** as output by **Clean**.

```

file          := module_list '*0'
|             '*0'
module_list   := module module_list
|             module
module        := function_module
function_module := header function '*0'
header        := function_name declarations 'H'
function_name  := '(' ID
declarations   := arguments prolog non_arg
arguments     := argument arguments
|

```

```

non_arg          := scope_stmts non_arg
                  | string_definition non_arg
                  | twit_string_definition non_arg
                  | constant_definition non_arg

prolog           := 'P' min_args max_args RELOC

scope_stmts      := scope_statement scope_stmts
                  |

scope_statement  := auto_var_declaration
                  | auto_vector_declaration
                  | auto_string_declaration
                  | extrn_var_declaration

extrn_var_declaration := 'g' REFERENCE name_id

function         := statement_list '*0'

statement_list   := statement statement_list
                  |

statement        := label_definition
                  | jump_statement
                  | expression
                  | return_statement
                  | word_select
                  | string_select
                  | jump_false_statement
                  | jump_true_statement
                  | twit_statement
                  | disable
                  | enable

string_definition := ''' REFERENCE string_id

twit_string_definition := ''' REFERENCE string_id

jump_statement   := 'J' label

label_definition := 'l' label

expression       := 'E' reference_list '*0'

return_statement := 'R' possible_reference_list '*0'

word_select      := w_sel_stmt w_case_list end_w_sel

```

```

string_select      := s_sel_stmt s_case_list end_s_sel
w_sel_stmt         := 'W' reference_list '*0'
s_sel_stmt         := 'S' reference_list '*0'
w_case_list        := word_case_statement w_case_list
                   | word_case_statement
str_case_list      := str_case_statement str_case_list
                   | str_case_statement
word_case_statement := word_case statement_list
word_case          := simple_word_case
                   | range_word_case
                   | default_case
str_case_statement := str_case statement_list
str_case           := 'b' string_id
                   | default_case
reference_list      := REFERENCE reference_list
                   | REFERENCE
possible_reference_list := reference_list
                   |
jump_false_statement := 'F' label reference_list '*0'
jump_true_statement  := 'T' label reference_list '*0'
twit_statement       := 't' reference_list '*0'
argument            := 'a' REFERENCE
end_of_header        := 'H'
auto_var_declaration := 'A' REFERENCE
auto_vector_declaration := 'V' REFERENCE CONSTANT
auto_string_declaration := 'v' REFERENCE CONSTANT
name_id             := ID

```

|                     |    |                        |
|---------------------|----|------------------------|
| string_id           | := | ID                     |
| constant_definition | := | 'c' REFERENCE CONSTANT |
| label               | := | REFERENCE              |
| end_w_sel           | := | 'w'                    |
| end_s_sel           | := | 's'                    |
| simple_word_case    | := | 'i' CONSTANT           |
| range_word_case     | := | 'r' CONSTANT CONSTANT  |
| default_case        | := | 'D'                    |
| disable             | := | 'd'                    |
| enable              | := | 'e'                    |
| min_args            | := | BYTE                   |
| max_args            | := | BYTE                   |

## Appendix B

### Code table functions for TI 990

Arith\_expr\_gen( tree )

```
\ This recursive function walks the
\ expression tree Left-to-Right (depth first)
\ generating TI machine code for evaluating
\ the expression
```

```
{
  extrn Last_nargs, Nargs_passed, Nargs_locn;
  extrn Right, Left, Destination;
  extrn Action_stack_ptr, Argument_ptr, Scratch;
  extrn Action_stack;
  auto destination, p, label, op, done;

  if( !((op=OP[tree]) OPERATOR) ) return;
  if( op == IL_ARG_MARK ) return;

  select( op )
  {
    case IL_QUERY:
    {
      label = Tree_label();
      Logical_expr_gen( LEFT[tree], label, FALSE );

      Arith_expr_gen( LEFT[RIGHT[tree]] );
      destination = RESULTIN[tree];
      Left = Operand( LEFT[RIGHT[tree]] );
      p = TYPE[Left] & TYPE_FIELD;
      if( !Is_in(Left, destination) )
        if( p==CONST_TYPE || p==STRING_TYPE )
          if( p==CONST_TYPE &&
              CONST_VALUE[Left]==0 )
            Code( ".CLR.", destination );
          else if( p==CONST_TYPE &&
                   CONST_VALUE[Left]==-1 )
            Code( ".SET0.", destination );
          else if( Reg(destination) )
            Code( ".LI.", destination, Left );
          else Code( ".MOV.", destination, Left );
        else Code( ".MOV.", destination, Left );
      Hop_gen( ".JMP.", done=Tree_label() );
    }
  }
}
```

```

        Backplug_gen( label );
        Last_nargs = -1;
        Arith_expr_gen( RIGHT[RIGHT[tree]] );
        Right = Operand( RIGHT[RIGHT[tree]] );
        p = TYPE[Right] & TYPE_FIELD;
        if( !Is_in(Right, destination) )
            if( p==CONST_TYPE || p==STRING_TYPE )
                if( p==CONST_TYPE &&
                    CONST_VALUE[Right]==0 )
                    Code( ".CLR.", destination );
                else if( p==CONST_TYPE &&
                    CONST_VALUE[Right]==-1 )
                    Code( ".SET0.", destination );
                else if( Reg(destination) )
                    Code( ".LI.", destination, Right );
                else Code( ".MOV.", destination, Right );
            else Code( ".MOV.", destination, Right );
        Backplug_gen( done );
        Last_nargs = -1;
        return;
    }
case IL_EQU:
case IL_NEQU:
case IL_GTE:
case IL_LTE:
case IL_GT:
case IL_LT:
case IL_LOGOR:
case IL_LOGAND:
case IL_NOT:
    {
        destination = RESULTIN[tree];
        label = Tree_label();
        Code( ".CLR.", destination );
        Logical_expr_gen( tree, label, FALSE );
        Code( ".INC.", destination );
        Backplug_gen( label );
        return;
    }
}

Arith_expr_gen( LEFT[tree] );

if( op NOT_UNARY )
{
    Arith_expr_gen( RIGHT[tree] );
    Right = Operand( RIGHT[tree] );
}

```



```

    }
    Left = Operand( LEFT[tree] );
    Destination = RESULTIN[tree];
    Scratch = SCRATCH[tree];

    if( op == IL_NFCTN_CALL || op == TI_FETCH_NFCTN_VAL )
    {
        Nargs_passed = TRUE;

        p = CONST_VALUE[Right];
        if( p == 0 )
        {
            if( Last_nargs != 0 ) Code( .CLR., Nargs_locn );
        }
        else if( Last_nargs != -1 )
            select( p - Last_nargs )
            {
                case -2: Code( .DECT., Nargs_locn );
                case -1: Code( .DEC., Nargs_locn );
                case 0: ;
                case 1: Code( .INC., Nargs_locn );
                case 2: Code( .INCT., Nargs_locn );
                default: Code( .MOV., Nargs_locn, Right );
            }
        else Code( .MOV., Nargs_locn, Right );
        Last_nargs = p;
    }

    Action_stack_ptr = Action_stack;
    Argument_ptr = Action_stack;
    Test_and_act( op, 0 );
}

```

```
Code( op; left_oper, right_oper )
```

```
\ Output one TI instruction. It may be more than one
\ word in length. Right_oper and left_oper are pointers
\ to symbol table entries. "left" and "right" refer to
\ the order in which the operands appear in the machine
\ instruction.
```

```
{
  extrn Counter;
  auto disp, lop, rr;

  Check_addressability();

  select( .Nargs() )
  {
    case 2: if( rr=Reg(left_oper) ) left_oper = rr;

    case 3:
    {
      if( rr=Reg(left_oper) ) left_oper = rr;
      if( rr=Reg(right_oper) ) right_oper = rr;
    }
  }

  select( op )
  {
    case .A. :
    case .AB. :
    case .C. :
    case .CB. :
    case .S. :
    case .SB. :
    case .SOC. :
    case .SOCB.:
    case .SZC. :
    case .SZCB.:
    case .MOV. :
    case .MOVB.:
    {
      Load_word( op | (ADDRMOD[left_oper]<<6) |
                  ADDRMOD[right_oper] );
      Cond_addr_gen( right_oper, FALSE );
      Cond_addr_gen( left_oper, FALSE );
    }
    case .COC. :
    case .CZC. :
    case .XOR. :
    case .MPY. :
```

```

case .DIV. :
{
    Load_word( op | (ADDRMOD[left_oper]<<6) |
                ADDRMOD[right_oper] );
    Cond_addr_gen( right_oper, FALSE );
}
case .B. :
case .BL. :
case .BLWP.:
case .CLR. :
case .SETO.:
case .INV. :
case .NEG. :
case .ABS. :
case .SWPB.:
case .INC. :
case .INCT.:
case .DEC. :
case .DECT.:
case .X. :
case .LDS.:
case .LDD.:
{
    Load_word( op | ADDRMOD[left_oper] );
    Cond_addr_gen( left_oper, TRUE );
}
case .LDCR.:
case .STCR.:
case .XOP. :
{
    Load_word( op | ADDRMOD[right_oper] |
                (CONST_VALUE[left_oper]<<6) );
    Cond_addr_gen( right_oper, FALSE );
}
case .SBO. :
case .SBZ. :
case .TB. :
{
    lop = CONST_VALUE[left_oper];
    disp = lop>=0 ? lop&0xFF : (~-lop)&0xFF + 1;
    Load_word( op | disp );
}
case .JEQ. :
case .JGT. :
case .JH. :
case .JHE. :
case .JL. :
case .JLE. :

```

```

case .JLT. :
case .JMP. :
case .JNC. :
case .JNE. :
case .JNO. :
case .JOC. :
case .JOP. :
    {
        disp = (VALUE[left_oper] - *Counter - 2)/2;
        disp = disp >= 0 ? disp : (~-disp) & $FF + 1;
        Load_word( op | disp );
    }
case .SLA. :
case .SRA. :
case .SRC. :
case .SRL. :
case .LMF. :

        Load_word( op | ADDRMOD[right_oper] |
                    (CONST_VALUE[left_oper] << 4) );

case .AI. :
case .ANDI.:
case .CI. :
case .LI. :
case .ORI. :
    {
        Load_word( op | ADDRMOD[left_oper] );
        Cond_addr_gen( right_oper, TRUE );
    }
case .LWPI.:
case .LIMI.:
    {
        Load_word( op );
        Cond_addr_gen( left_oper, TRUE );
    }
case .STST.:
case .STWP.:

        Load_word( op | ADDRMOD[left_oper] );

case .RTWP.:
case .IDLE.:
case .RSET.:
case .CKON.:
case .CKOF.:
case .LREX.:

```

```
        Load_word( op );  
    default:  Error( " invalid opcode in a twit" );  
    }  
}
```

```
Cond_addr_gen( operand, immediate )
```

```
\ Called from Code and Twit_gen to, when necessary
\ output an instruction address.. If the operand is
\ a constant, the value from a literal pool is used
\ (and put into a pool if none exists) unless the
\ immediate argument is nonzero. In such cases the
\ immediate operand of the instruction is generated
\ and becomes part of the pool.
```

```
{
    extrn Counter;
    auto type, rel_wsd, value;

    if( (ADDRMOD[operand]&MODE_BITS) != INDEXED_MODE )
        return;

    type = TYPE[operand] & TYPE_FIELD;
    rel_wsd = TYPE[operand] & REL_WSD_FIELD;
    value = Raw_value( operand );

    if( type == CONST_TYPE || type == STRING_TYPE )
        if( immediate )
        {
            TYPE[operand] = type |
                (ADDR_RELDISC<<8) |
                WSD_CODE |
                SYMBOL_DEFINED;
            VALUE[operand] = *Counter;
            if( type == CONST_TYPE ) rel_wsd = 0;
        }
        else if( !(TYPE[operand]&SYMBOL_DEFINED) )
        {
            Set_loc( WSD_DATA );
            VALUE[operand] = *Counter;
            if( rel_wsd == 0 )
                Load_word( value );
            else
                Rload_word( rel_wsd>>8,
                    rel_wsd&WSD_FIELD,
                    value );
            rel_wsd = ADDR_RELDISC<<8 | WSD_DATA;
            Set_loc( WSD_CODE );
            TYPE[operand] = type | rel_wsd | SYMBOL_DEFINED;
            value = VALUE[operand];
        }
        else value = VALUE[operand];

    if( rel_wsd == 0 )
```

```
        Load_word( value );  
    else  
        Rload_word( rel_wsd>>8, rel_wsd&WSD_FIELD, value );  
}
```

Expand( operand )

```

\ Return the address of the operand.
{
  extrn Left, Right, Destination, Scratch, R;
  extrn Temp_1, Temp_2, Temp_3, Temp_4, Temp_5, Constant_16;
  extrn Action_stack_ptr, Argument_ptr, Return_val_locn;
  extrn Const_one, Constant_2, Constant_4, Constant_8;

  select( operand )
  {
    case left:      return( &Left );
    case right:     return( &Right );
    case dest:      return( &Destination );
    case scratch:   return( &Scratch );
    case const_1:   return( &Const_one );
    case value_1:   return( Const_one+CONST_VALUE );
    case const_2:   return( &Constant_2 );
    case const_4:   return( &Constant_4 );
    case const_8:   return( &Constant_8 );
    case value_8:   return( Constant_8+CONST_VALUE );
    case const_16:  return( &Constant_16 );
    case val_left:  return( Left+VALUE );
    case const_val_right: return( Right+CONST_VALUE );
    case val_dest:  return( Destination+VALUE );
    case r11:       return( R+11 );
    case r12:       return( R+12 );
    case temp1:     return( &Temp_1 );
    case temp2:     return( &Temp_2 );
    case temp3:     return( &Temp_3 );
    case temp4:     return( &Temp_4 );
    case temp5:     return( &Temp_5 );
    case stk1:      return( Action_stack_ptr );
    case stk2:      return( Action_stack_ptr+1 );
    case stk3:      return( Action_stack_ptr+2 );
    case arg1:      return( Argument_ptr );
    case arg2:      return( Argument_ptr+1 );
    case arg3:      return( Argument_ptr+2 );
    case return_val_locn: return( &Return_val_locn );
    case addrmod_dest: return( Destination+ADDRMOD );
  }
}

```



```

Test( operator, op1 )
{
    extrn Argument_ptr, Destination, Left, Right;
    auto type;

    type = TYPE[op1]&TYPE_FIELD;
    select( operator )
    {
        case const_or_str: return( type==STRING_TYPE ||
                                   type==CONST_TYPE );

        case in_reg:      return( Reg( op1 ) != 0 );

        case is_arg1:     return( op1 == *Argument_ptr );

        case is_arg2:     return( op1 == Argument_ptr[1] );

        case is_auto:     return( type==AUTO_TYPE );

        case is_const_0:   return( type==CONST_TYPE &&
                                   CONST_VALUE[op1] == 0 );

        case is_const_1:   return( type==CONST_TYPE &&
                                   CONST_VALUE[op1] == 1 );

        case is_const_2:   return( type==CONST_TYPE &&
                                   CONST_VALUE[op1] == 2 );

        case is_constant: return( type==CONST_TYPE );

        case is_extrn:     return( type==EXTRN_TYPE );

        case is_gt_0:      return( type==CONST_TYPE &&
                                   CONST_VALUE[op1] > 0 );

        case is_gt_15:     return( type==CONST_TYPE &&
                                   CONST_VALUE[op1] > 15 );

        case is_in_arg1:   return( Is_in(op1, *Argument_ptr) );

        case is_in_dest:   return( Is_in(op1, Destination) );

        case is_left:      return( op1 == Left );

        case is_neg_1:      return( type==CONST_TYPE &&
                                   CONST_VALUE[op1] == -1 );

        case is_right:     return( op1 == Right );
    }
}

```

```
    case is_string: return( type==STRING_TYPE );  
    case volatile: return( Volatile( op1 ) );  
    default:      return( TRUE );  
  }  
}
```

```

Action( operator; op1, op2, op3 )
{
    extrn R;
    select( operator )
    {
        case add:    *op1 += *op2;
        case backplug: Backplug_gen( *op1 );
        case code_var: Code( *Expand( op3 ), *op1, *op2 );
        case copy:   *op1 = *op2;
        case constant: *op1 = Constant( *op2 );
        case register: *op1 = R[*op2];
        case def_label: *op1 = Tree_label();
        case get_reg:   *op1 = R[ADDRMOD[*op2]&$F];
        case reg_num:   *op1 = .ADDRMOD[Reg(*op2)];
        case hop:      Hop_gen( op3, *op1 );
        case neg:      *op1 = -*op1;
        case compliment: *op1 = ~*op1;
        case set:      *op1 = op3;
        case shift: (op2=*op2)<0 ? *op1>>=-op2 : (*op1<<=op2);
        case sub:      *op1 -= *op2;
    }
}

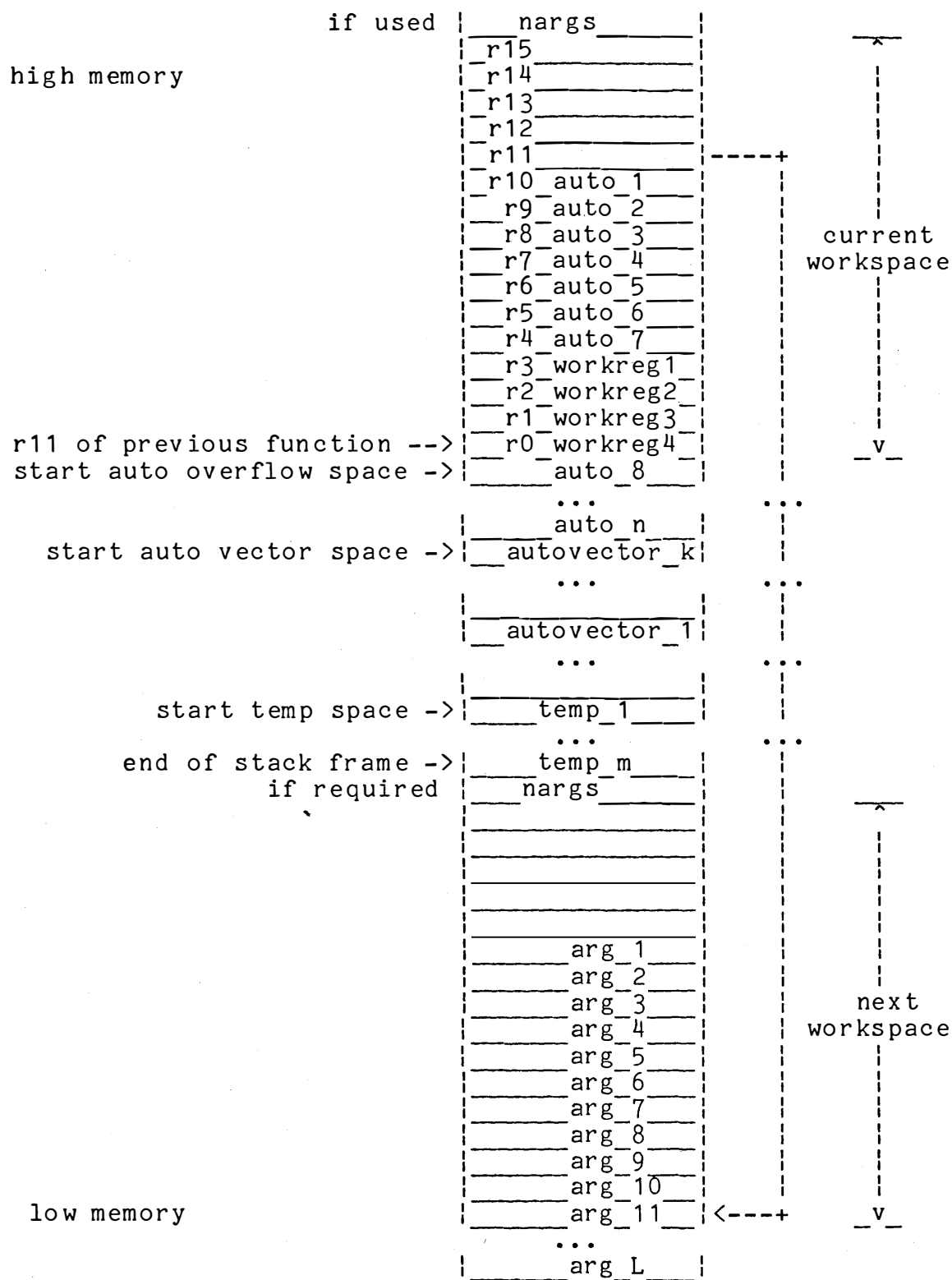
```

## Appendix C

### Stacks on the TI 990

The implementation of stacks on the TI 990 makes use of the Workspace concept. A Workspace is a 16-word area in memory which constitutes 16 registers. A hardware register called the Workspace pointer can be modified to point to a different area, thus changing the register set currently in use. Upon first use of a new Workspace, certain registers are initialized to record the location of the previous Workspace and the processor status at the time the Workspace came into use. Arguments are passed to functions by setting the register contents of a soon to be used Workspace. When the function is called, the Workspace pointer is changed to use the new Workspace and the arguments will then reside in the registers used by the function.

The diagram below illustrates the structure of stack frames in a stack which is grown downwards in memory (toward location 0). For purposes of this discussion, function arguments and auto variables are simply named auto 1 through auto n (where  $n = \text{number of formal args} + \text{number of autos}$ ).



Five extra locations are reserved for registers 11 through 15 between the start of the stack frame and the first auto. Therefore, regardless of the number of formal arguments or autos, each function requires a minimum of 5 words per stack frame.

Fortunately, the above diagram represents somewhat of a worst case example. If there are no auto vectors defined in the function, the associated stack space does not exist. If the number of autos is less than 8 the auto overflow space is not required. Usually the temp space is not required, since 4 work registers are sufficient to hold temporaries created by reasonably complex expressions.

In general, more work registers become available as fewer autos are defined. Work registers are allocated starting at the first unused auto register, so that if not all work registers are required, what remains of that part of the workspace becomes part or all auto vector space (if any). In such a case, if no auto vector space is defined, the current workspace and the next workspace will overlap by the number of registers not used by the function.

A total of 11 registers are available for auto variables and temporaries, of which at most 7 may be auto variables. If more autos are defined, the remainder are referenced by indexing with register 11 to the auto overflow

area. Similarly, if the function requires more temporaries than can be accomodated (i.e. greater than 11 minus the number of autos) the remainder are referenced by indexing with register 11 to the temp area in the stack frame. If there are more than 7 arguments (this does not include the ? formal argument type of function discussed below) the remaining arguments are copied to the auto overflow area.

During the execution of a function, register 11 is used as a pointer to a 16 word block in the stack frame which will be used as the workspace for all functions called by it. Therefore, referencing or setting arguments for the next function call is a matter of indexing with register 11. Register 12 is used as a special temporary for such things as executing select statements. It is also used to hold the address of a function being called. Returned values are kept in register 12 of the called function, so that the function using the value can find it by indexing to it with register 11. Functions are called by executing a .BLWP. R11, causing the next workspace to become the current workspace. The function which has been called first executes a common prolog routine which stores the new workspace pointer in register 11 and increments it to point to the next workspace after first checking that the increment will not cause the stack to overflow. The increment value, determined at

compile time and stored in the word immediately following the call to the common prolog routine, is the sum of the auto overflow space, the temp space, the auto vector space, and the 16-word block which will become the next workspace (17 if any functions called require nargs), less the number of work registers not required by the function.

For functions defined with the '?' formal argument list, an interim stack frame is first created which contains only the arguments, before the common prolog routine is executed.

Return from any Eh function is accomplished by executing the .RTWP. instruction which re-instates the previous workspace and stack frame.



## Appendix D

### Eh Implementation on the Nova

#### Register Usage:

Ac2 is always used as the stack pointer. In general, the result of an expression may occur in Ac0, Ac1 or Ac3. The expression involved in return and select statements, however, always leaves its result in Ac0 as do all function calls. The contents of registers are forgotten from statement to statement.

#### Stack Usage:

At any particular time in the execution of a program, there exists a current function and a current stack frame (see Figure 1). The stack frame is of fixed size, that is, throughout the execution of a function the size of the frame remains constant. Additionally, if the function is not a '?' function, the frame size is constant over all invocations of the function. For '?' functions the frame size varies with each invocation, depending on the number of arguments passed to it. The current stack frame includes the current function's arguments, autos, auto vector space and temporaries as well as the stack pointer of the current function's caller (the father function). The current return

address is kept in the father's stack frame (at 0[father's stack pointer]). If the current function requires a count of the number of arguments passed to it (Nargs), it is also found in the father's stack frame (at 2[father's stack pointer]). Note that new stack frames are created by moving the stack pointer towards memory location 0, that is, the stack "grows" downwards in memory.

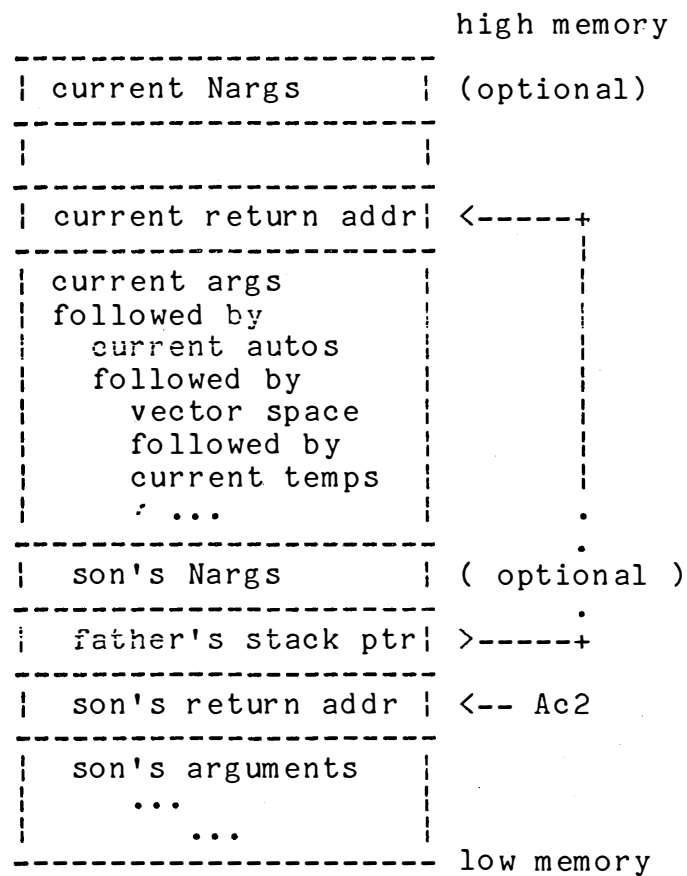


Figure 1  
Stack Frame Layout

The current stack frame, therefore, also reserves a

word for the return address of functions called by the current function (son functions) and optionally reserves a word for the son function's Nargs if at least one of the son functions requires it.

#### **Function Layout:**

The stack frame layout in Figure 1 has two distinct advantages over previously used stack frames. Firstly, the stack pointer decrement does not need to be passed to a son function when the son function is called. Each function decrements the stack pointer once when execution begins and the decrement value depends only on the amount of stack space required by its autos and temporaries. In the case of a '?' type function, the Nargs value is also included in the stack decrement value.

The stack decrementing operation (as well as stack overflow checking) is performed by the library routine `.Fcn_prolog`, or `.Qfcn_prolog` if the current function is of '?' type. In either case the stack decrement follows the call to the prolog routine:

```

fcn_entry:  sta      3, 0, 2      \ save return address
           jsr      .Fcn_prolog  \ in father's frame
           (decrement)          \ decrement and check
           .                  \ the stack pointer
           .                  \ rest of function
           .
           lda      2, 1, 2      \ restore old stack ptr
           jmp      @0, 2        \ return to father

```

A return to the father function is achieved by simply restoring the father's stack pointer and jumping indirect through the return location in the father's stack frame.

The second advantage of the stack layout is the fact that both Nargs and arguments ('?' or otherwise) are within easy reach of the current function. This is due primarily to the fact that the father's stack pointer is saved in the current frame.

The .Nargs and .Arg functions follow:

```

.Nargs:  sta      3, 0, 2      \ store return address
         lda      3, 1, 2      \ pick up old stack ptr
         lda      0, 2, 3      \ pick up Nargs
         jmp      @0, 2        \ return

.Arg:    sta      3, 0, 2      \ store return address
         lda      3, 1, 2      \ pick up old stack
         lda      0, -1, 2     \ pick up arg number
         sub      0, 3         \ calculate arg address
         lda      0, 0, 3      \ load the argument
         jmp      @0, 2        \ return to caller

```

The prolog routines are in fact one routine with two entry points. The .Qfcn\_prolog entry just has the added task of altering the stack decrement value by the value of Nargs

before continuing as if it were a normal function.

```
.Qfcn_prolog:
    lda    0, 0, 3    \ pick up the decrement
    lda    1, 2, 2    \ pick up Nargs
    sub    1, 0       \ the real decrement
    jmp    .+2        \ continue

.Fcn_prolog:
    lda    0, 0, 3    \ pick up decrement
    mov    2, 1       \ save old stack ptr
    add    0, 2       \ bump stack pointer
    lda    0, .Active_stack_top

    adcz#  0, 2, SZC  \ stack overflow ?
    jmp    help       \ yes
    sta    1, 1, 2    \ store old stack ptr
    jmp    1, 3       \ return to called fcn
```

### Function calling:

Function calling consists of passing arguments and performing a jump subroutine instruction. To pass a value to the *i*-th argument of the son function, the value is simply stored at *-i, 2*. For example, assuming that *z* is an auto with displacement 5 on the stack, the statement

```
z = Function( 1, 3 );
```

translates into

```
    subzl  0, 0       \ generate a 1
    sta    0, -1, 2    \ save in argument 1
    lda    0, c_3      \ get second argument
    sta    0, -2, 2    \ save in argument 2
    jsr    @F_addr     \ call the function
    sta    0, 5, 2     \ save the return value
    .
    .
    \ constant pool
c_3:      .dc1        3
F_addr:   .dc1        Function
```

If Function requires Nargs, one of several ways of passing Nargs is used depending on the previously passed Nargs value. Note that with a fixed size stack frame the location of Nargs remains constant. The first Nargs value is passed by loading it into a register and storing it in the son's Nargs location. If the Nargs value is the same as the previous Nargs value no Nargs passing code is required since the previous value is still there. If the Nargs value is non-zero and differs by one from the value previously passed, it may be set by a simple `isz` or `dsz` instruction. Otherwise a load/store combination is used to set the Nargs value.